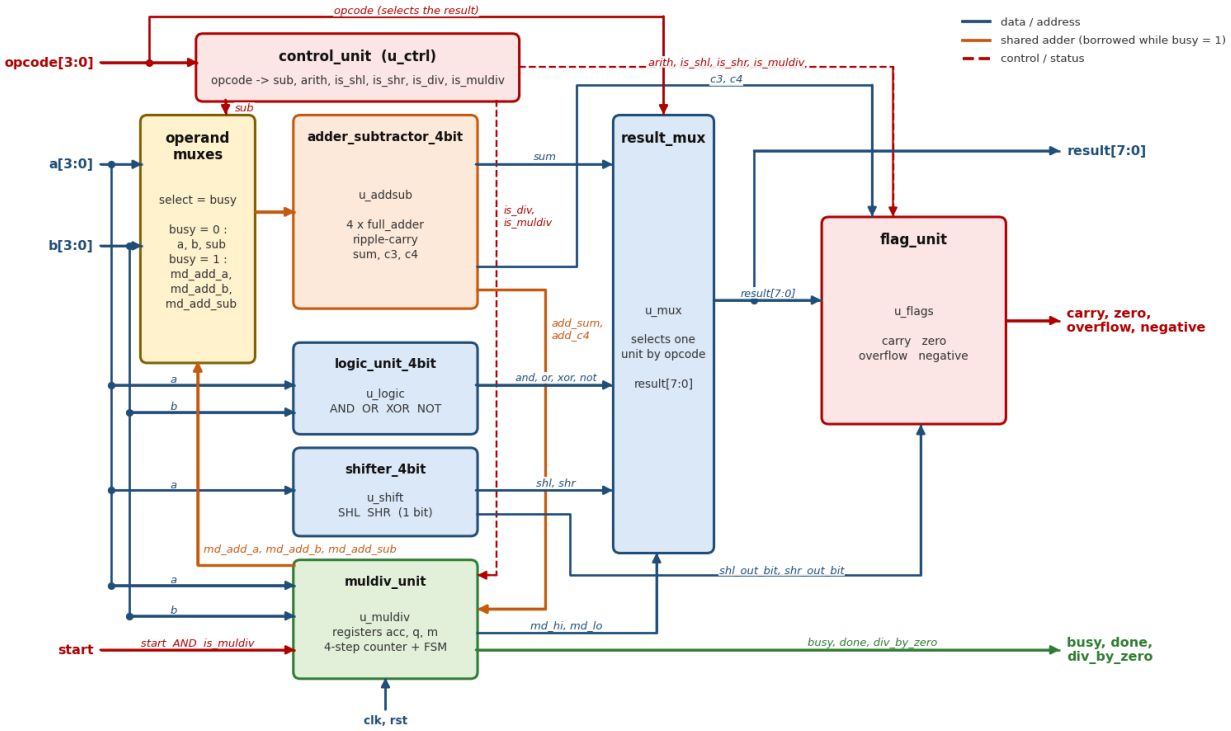
PROJECT 01

DESIGN AND VERIFICATION 4-BIT ALU



Student: Truong Dong Hung

Roll No: SE201121

Ho Chi Minh City, October 2026

Abstract

This report presents a 4-bit Arithmetic Logic Unit (ALU) written in Verilog HDL and implemented in Xilinx Vivado 2018.3. The ALU performs ten operations: the eight single-cycle operations ADD, SUB, AND, OR, XOR, NOT, SHL and SHR, together with two five-cycle operations, MUL and DIV, producing an 8-bit result and five status flags. Multiplication and division are carried out by shift-and-add and shift-and-subtract, reusing the ALU's own adder/subtractor rather than instantiating a separate multiplier array, which keeps the circuit small at the cost of taking several clock cycles instead of one. The design is split into nine Verilog source files, one module per function, and is verified by a self-checking testbench that exercises all 4,096 possible opcode and operand combinations as well as a set of protocol scenarios, ending in a printed PASS line with zero errors. This document walks through every module with its function, its truth table taken from simulation, the corresponding Vivado waveform and its complete Verilog source, followed by the signal connections between modules, the file hierarchy, and the Vivado flow from source entry to utilisation reporting (screenshots F1 to F6). The implemented design occupies 62 Slice LUTs and 18 Slice registers on an Artix-7 device.

Contents

1. Introduction 5
1.1. Objectives 5
1.2. Scope 5
2. Theory in Brief 6
2.1. Carry versus Overflow 6
2.2. Multiply as Shift-and-Add 6
2.3. Divide as Shift-and-Subtract (Restoring) 6
2.4. Why Sharing the Adder Matters 6
3. System-Level Design 8
3.1. Top-Level Interface 8
3.2. Opcode Map 8
3.3. Top-Level Diagram 8
3.4. File Hierarchy 10
3.5. Signal Connections Between Modules 10
3.6. Operating Rules for MUL and DIV 11
4. Design, Truth Tables, Waveforms and Source Code 13
4.1. control_unit - Opcode Decoder 13
4.2. full_adder - One-Bit Adder 14
4.3. adder_subtractor_4bit - Shared Adder / Subtractor 16
4.4. logic_unit_4bit - Bitwise Operations 18
4.5. shifter_4bit - Logical Shift 19
4.6. muldiv_unit - Sequential Multiplier / Divider (Focus Module) 20
4.7. result_mux - Final Result Selection 24
4.8. flag_unit - Status Flags 26
4.9. alu_4bit - Top Level 27
5. Vivado Flow - F1 to F6 30
5.1. F1 - Sources Window 30
5.2. F2 and F3 - Behavioural Simulation 30
5.3. F4 - Elaborated Schematic 32
5.4. F5 - Synthesised Schematic 33
5.5. F6 - Utilisation Report 33
6. Testbench and Verification 35
6.1. Strategy 35
6.2. Protocol Scenarios (Section 2.4) 36
6.3. Verification Results (Actual Run) 36

6.4. Does the Testbench Catch Bugs? 36
6.5. Complete Testbench Source 37
7. Synthesis, Implementation, Results 43
7.1. Utilisation and Power 43
8. Flow Explanation and Conclusion 44
8.1. Flow 44
8.2. Conclusion 44

1. Introduction

The Arithmetic Logic Unit (ALU) is the part of a processor that carries out every arithmetic and logic calculation an instruction asks for, while the rest of the processor exists mainly to feed it operands and store what it produces. This project designs a 4-bit ALU in Verilog that goes beyond the usual eight combinational operations by adding multiplication and division, without building a dedicated multiplier or divider array. Instead, MUL and DIV reuse the ALU's own adder over several clock cycles through shift-and-add and shift-and-subtract, which is the standard area-saving technique used inside small processors and microcontrollers.

1.1. Objectives

The objectives of this project are to design a 4-bit ALU with ten operations in modular Verilog, with one sub-module per function; to implement MUL and DIV by shifting and reusing the shared adder/subtractor rather than adding a second arithmetic unit; to verify the design exhaustively with a self-checking testbench covering all 4,096 opcode and operand combinations in addition to a set of protocol scenarios; and to simulate, synthesise and implement the design in Vivado, reporting its resource utilisation on an Artix-7 target.

1.2. Scope

Item	Value
Language / Tool	Verilog HDL (IEEE 1364-2001), Xilinx Vivado 2018.3
Target device	Artix-7, xc7a35tcpg236-1
Operands / Result	a, b: 4-bit each; result: 8-bit (holds a full product)
Operations	ADD, SUB, AND, OR, XOR, NOT, SHL, SHR (1 clock); MUL, DIV (5 clocks)
Flags / Handshake	carry, zero, overflow, negative, div_by_zero; busy, done
Verification	alu_4bit_tb.v: all 4,096 opcode / operand combinations (3,584 single-cycle + 512 MUL/DIV) plus protocol scenarios, 30,563 comparisons, ending in a printed PASS line
Implementation result	62 Slice LUTs, 18 Slice registers, 19 Slices (Vivado 2018.3)

Table 1. Project summary

2. Theory in Brief

2.1. Carry versus Overflow

Four bits can hold sixteen patterns. Read as unsigned numbers they represent 0 to 15, while read as two's complement signed numbers the top bit carries a weight of -8, giving the range -8 to +7. The same adder circuit produces the correct bit pattern for both interpretations; only the meaning given to the result and to the flags differs. Subtraction is implemented as addition: A - B equals A plus the bitwise complement of B plus one, so each bit of B is passed through an XOR gate controlled by a signal called sub, and sub is also fed into the adder's carry-in to supply the extra one. One adder therefore performs both ADD and SUB. Carry, the fifth bit that falls out of a 4-bit addition, tells whether the unsigned answer exceeded 15, and for subtraction a carry of 1 means no borrow was needed. Overflow instead tells whether the signed answer fell outside -8 to +7. For A + B this happens when both operands have the same sign and the result has the other sign; for A - B it happens when the operands have different signs and the result's sign differs from that of A. In hardware both cases are detected simply as C3 XOR C4, the carry into the sign bit differing from the carry out of it.

2.2. Multiply as Shift-and-Add

Binary long multiplication adds the multiplicand, shifted according to each multiplier bit, for every bit that is 1. The sequential version performs exactly this, one row per clock cycle, using a single shared adder and three 4-bit registers named acc, q and m. On each of four steps, if the lowest bit of q is 1 the multiplicand m is added into acc, and then the combined register {carry, acc, q} is shifted one place to the right; the shift simultaneously brings the next multiplier bit into q[0] and moves the completed low bit of the product into the top of q. After four steps the full 8-bit product is held in {acc, q}.

2.3. Divide as Shift-and-Subtract (Restoring)

Binary long division shifts in one dividend bit at a time and asks whether the divisor fits into the value accumulated so far. The restoring algorithm shifts {acc, q} one place to the left, then attempts to subtract the divisor m from the shifted accumulator; if the subtraction succeeds without a borrow the divisor fit, so the result is kept and the new quotient bit is set to 1, and otherwise the accumulator is left unchanged (restored) and the quotient bit is set to 0. After four steps the quotient occupies q and the remainder occupies acc. When the divisor is zero the subtraction always appears to fit, so the algorithm naturally produces a quotient of all ones and a remainder equal to the dividend, and the unit additionally raises a div_by_zero flag so the condition can be detected explicitly.

In the RTL the decision is fits = r_top OR (no borrow), where r_top is the bit shifted out of the top of acc. For a 4-bit dividend r_top is always 0: before step k only k - 1 bits of the dividend have entered acc, so acc is at most 7 before every step. This was confirmed by simulating all 256 operand pairs: r_top never became 1 and the largest acc was 7. The term is therefore a safeguard that would only matter for a dividend wider than the accumulator; for this ALU, fits simply means that the subtraction produced no borrow.

2.4. Why Sharing the Adder Matters

A fully combinational multiplier or divider needs on the order of n-squared gates. Performing one step per clock with a single n-bit adder instead grows only linearly with n, at the cost of several clock cycles rather

than one. Because MUL needs an adder and DIV needs a subtractor, and the ALU already contains exactly that circuit, the sequential multiplier/divider unit, muldiv_unit, contains no adder of its own. While it is active, three multiplexers in the top-level module redirect the shared adder's inputs away from the external operands a and b and toward the registers inside muldiv_unit, and the rest of the time the same adder serves ADD and SUB as normal. This hardware sharing is visible directly in the RTL schematic produced by Vivado.

3. System-Level Design

3.1. Top-Level Interface

The top module alu_4bit exposes a clock and an asynchronous active-high reset, a one-clock start pulse used only by MUL and DIV, the two 4-bit operands a and b, and a 4-bit opcode selecting the operation. Its outputs are the 8-bit result, the four arithmetic flags carry, zero, overflow and negative, the div_by_zero flag, and the two handshake signals busy and done. For every single-cycle opcode the result and flags are valid immediately, with no dependency on the clock, as long as no MUL or DIV is running (see Section 3.6). MUL and DIV treat a and b as unsigned numbers (0 to 15); ADD and SUB produce the same bit pattern for unsigned and two's-complement operands, and the flags report both interpretations.

3.2. Opcode Map

Opcode	Operation	result[7:0]	Cycles
0000	ADD	{0000, a+b}	1
0001	SUB	{0000, a-b}	1
0010 / 0011 / 0100 / 0101	AND / OR / XOR / NOT	{0000, ...}	1
0110 / 0111	SHL / SHR	{0000, ...}	1
1000	MUL	a x b (8-bit product)	5
1001	DIV	{remainder, quotient}	5
1010 - 1111	unused	00000000	-

Table 2. Opcode to operation mapping

Operation	result[7:0] range	Notes
ADD, SUB, logic, shifts	result[3:0] = 0 to 15, result[7:4] = 0	flags as in Section 4.8
MUL	0 to 225 (15 x 15)	carry = overflow = negative = 0; zero = 1 only when a = 0 or b = 0
DIV, b != 0	quotient 0 to 15, remainder 0 to 14 (remainder < b)	zero = 1 only when a = 0
DIV, b = 0	quotient 15 (1111), remainder = a	div_by_zero = 1
unused opcodes	0	zero = 1; carry, overflow, negative = 0

Table 3. Value ranges of the result and flag behaviour

3.3. Top-Level Diagram

alu_4bit instantiates seven sub-modules. All functional datapath units, the adder/subtractor, the logic unit, the shifter and the multiply/divide unit, compute on the same operands every cycle in parallel, and result_mux then selects the single output the opcode asked for. control_unit decodes the opcode once into a handful of one-bit control signals that the other blocks consume directly. Figure 1 is a simplified block diagram of this structure. The shared adder is drawn in orange because, while busy is high, its inputs are switched away from a and b and toward the registers of muldiv_unit by three multiplexers (add_in_a, add_in_b and add_sub in alu_4bit.v).

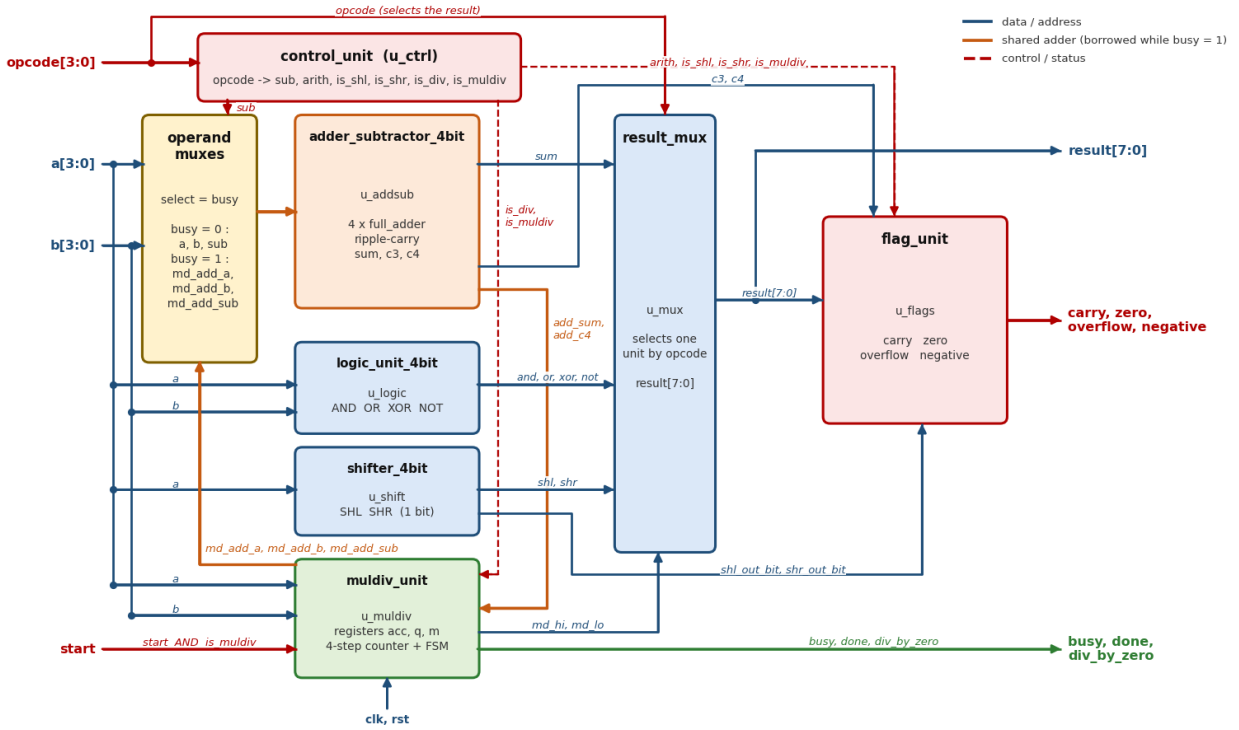


Figure 1. Simplified block diagram of alu_4bit (the orange path is the shared adder, borrowed by muldiv_unit while busy = 1).

Figure 2 is the corresponding elaborated schematic produced by Vivado. It names every signal exactly as it is declared in alu_4bit.v: opcode, a, b, clk, rst, start on the left; sub, arith, is_shl, is_shr, is_div, is_muldiv out of control_unit; add_in_a, add_in_b, add_sub into the shared adder; sum, c3, c4 out of it; and_out/or_out/xor_out/not_out out of the logic unit; shl/shr/shl_out_bit/shr_out_bit out of the shifter; md_add_a/md_add_b/md_add_sub, md_hi, md_lo, busy, done, div_by_zero on muldiv_unit; and result, carry, zero, overflow, negative as the final outputs.

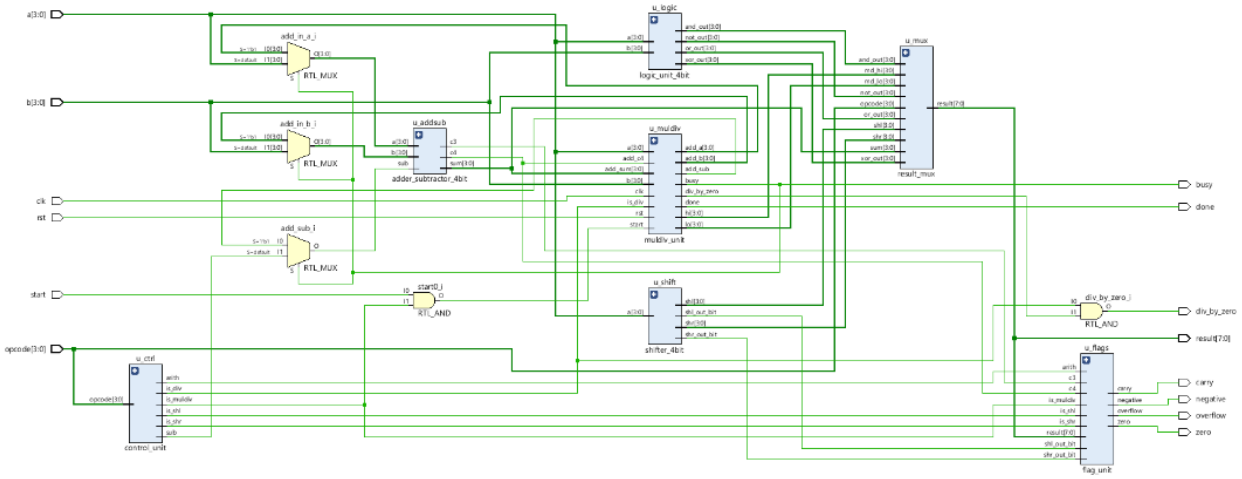


Figure 2. Vivado elaborated schematic of alu_4bit.

3.4. File Hierarchy

The design is split into one module per file, following the principle that each file has a single, clearly-named responsibility. alu_4bit.v is the top level: it instantiates every other module and contains no computation of its own beyond the three adder-sharing multiplexers. The testbench, alu_4bit_tb.v, instantiates alu_4bit and is a simulation-only file that is never synthesised into hardware. Figure 3 shows this instantiation tree graphically; the table below lists the same information with each module's instance name.

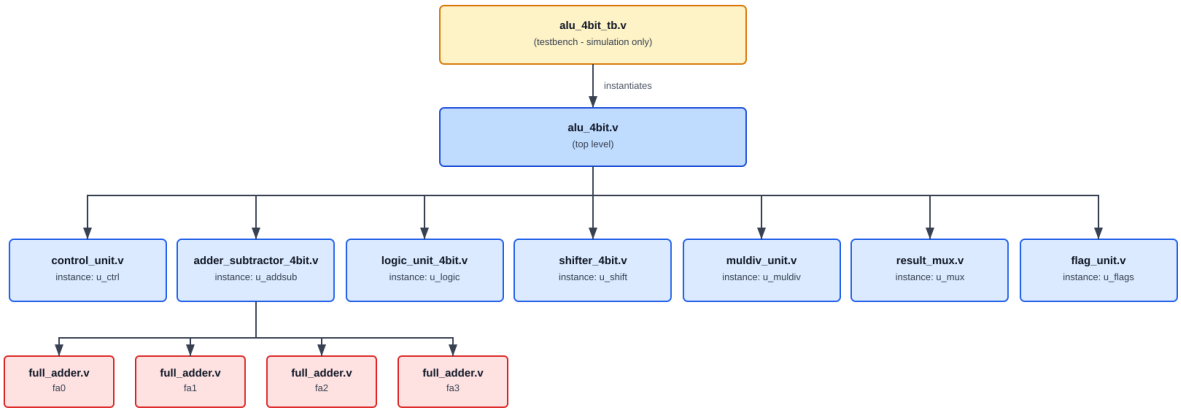


Figure 3. File hierarchy of the 4-bit ALU.

Reading this top to bottom: alu_4bit_tb.v drives alu_4bit.v, which in turn instantiates the seven functional modules; full_adder.v sits one level deeper still, instantiated four times inside adder_subtractor_4bit.v to form the ripple-carry chain. Every other module is instantiated exactly once, directly inside alu_4bit.v.

File	Module	Instance	Function
alu_4bit.v	alu_4bit	(top)	Instantiates all modules; adder-sharing multiplexers; start and div_by_zero gating
control_unit.v	control_unit	u_ctrl	Opcode decoder
adder_subtractor_4bit.v	adder_subtractor_4bit	u_addsub	Shared 4-bit adder / subtractor
full_adder.v	full_adder	fa0 .. fa3	One-bit adder, four copies inside u_addsub
logic_unit_4bit.v	logic_unit_4bit	u_logic	AND, OR, XOR, NOT
shifter_4bit.v	shifter_4bit	u_shift	Logical shift left / right by one
muldiv_unit.v	muldiv_unit	u_muldiv	Sequential multiplier / divider
result_mux.v	result_mux	u_mux	Selects the 8-bit result
flag_unit.v	flag_unit	u_flags	carry, zero, overflow, negative
alu_4bit_tb.v	alu_4bit_tb	dut	Self-checking testbench (simulation only)

Table 4. Modules, files and instance names

3.5. Signal Connections Between Modules

The table below lists every internal signal that crosses a module boundary: where it is produced, where it is consumed, and what it is for. This is the wiring diagram of the ALU expressed as a table rather than as a picture, and it is the fastest way to answer, during a defence, the question "how does signal X get from module A to module B".

Signal	Produced in	Used in	Purpose
opcode	testbench / port	control_unit, result_mux	selects the operation
a, b	testbench / port	adder (via mux), logic_unit, shifter, muldiv_unit	operands
sub	control_unit	alu_4bit -> adder_subtractor	0 = add, 1 = subtract
arith, is_shl, is_shr	control_unit	flag_unit	which flag rule applies
is_div, is_muldiv	control_unit	muldiv_unit, flag_unit, alu_4bit	start / mode / gating
md_add_a, md_add_b, md_add_sub	muldiv_unit	alu_4bit -> adder_subtractor	borrow the shared adder
busy	muldiv_unit	alu_4bit (mux select), port	who owns the adder / status
sum	adder_subtractor_4bit	result_mux, muldiv_unit	ADD/SUB result, MUL/DIV step
c4	adder_subtractor_4bit	flag_unit, muldiv_unit	carry / borrow
c3	adder_subtractor_4bit	flag_unit	overflow detection
and_out ... not_out	logic_unit_4bit	result_mux	logic results
shl, shr	shifter_4bit	result_mux	shift results
shl_out_bit, shr_out_bit	shifter_4bit	flag_unit	carry on shifts
md_hi, md_lo	muldiv_unit	result_mux	product / {remainder, quotient}
result	result_mux	flag_unit, port	final 8-bit answer
carry, zero, overflow, negative	flag_unit	port	status flags
done, div_by_zero	muldiv_unit / alu_4bit	port	status

Table 5. Signal connections between modules

3.6. Operating Rules for MUL and DIV

MUL and DIV follow a simple start / busy / done handshake. Put the opcode, a and b on the inputs, pulse start high for exactly one clock, and wait for done. On the rising clock edge that samples start, the operands are latched and busy goes high; four further rising edges perform the four steps, after which busy falls, done rises and result is final, so the operation takes 5 clock cycles in total.

Clock edge	Event	busy after the edge	done after the edge
0	start sampled: operands latched, registers loaded	1	0
1 to 3	steps 1 to 3	1	0
4	step 4: result final	0	1

Table 6. Timing of a MUL or DIV operation

The behaviour below is that of this RTL. Most of the rules are checked by the protocol scenarios of the testbench (Section 6); the last column says which ones. The rules marked "simulation only" were observed by simulating the RTL but are not part of the automated checks.

Situation	Behaviour	Checked by testbench
start pulsed while busy = 1	Ignored: the running operation is not disturbed	yes (2.4)
start pulsed with a non-MUL / DIV opcode	Ignored (start is gated with is_muldiv)	yes (2.4)
a or b changed after start	No effect: the operands were latched at start	yes (2.3)
ADD or SUB requested while busy = 1	Result is not valid: the adder is lent to muldiv_unit until done	simulation only
AND, OR, XOR, NOT, SHL, SHR while busy = 1	Work normally (they use their own units)	simulation only
result and done after a MUL / DIV	Held until the next start, as long as the opcode stays MUL / DIV	yes (2.4)
div_by_zero	Latched at start; the output is gated by opcode = DIV and stays set until the next start or reset	yes (2.4)
rst while busy = 1	Asynchronously aborts the operation: busy and done clear immediately	yes (2.4)
result and flags while busy = 1	Intermediate values ({acc, q} after each step, for example 11, 109, 158, 79 while computing 13 x 11): read them only when done = 1	simulation only
start held high for several clocks	Not a pulse: when an operation finishes the unit loads again at once, so a new operation starts every 5 clocks and done is high for one clock only	simulation only

Table 7. Behaviour of the handshake

4. Design, Truth Tables, Waveforms and Source Code

Each module below follows the same four-part structure, so that a reader, or the student presenting it, can move through the project one consistent way every time: (1) function - what the module does and why it exists; (2) truth table - the expected values, taken from simulating this exact RTL; (3) waveform - a Vivado simulation screenshot that shows the same values appearing on the hardware signals; and (4) source code - the complete Verilog of that module, exactly as stored in the rtl folder of the repository.

4.1. control_unit - Opcode Decoder

Function: control_unit decodes the 4-bit opcode once into six one-bit control signals, so every other block reads a simple control bit instead of comparing the opcode for itself. sub selects addition or subtraction inside the shared adder; arith marks ADD or SUB for the flag logic; is_shl and is_shr mark the two shift opcodes; is_div and is_muldiv gate the sequential multiply/divide unit and the result multiplexer.

Port	Direction	Description
opcode[3:0]	input	4-bit operation select
sub	output	1 for SUB (0001)
arith	output	1 for ADD or SUB
is_shl / is_shr	output	1 for SHL / SHR
is_div	output	1 for DIV (1001)
is_muldiv	output	1 for MUL or DIV

Table 8. control_unit - port list

opcode	Operation	sub	arith	is_shl	is_shr	is_div	is_muldiv
0000	ADD	0	1	0	0	0	0
0001	SUB	1	1	0	0	0	0
0010	AND	0	0	0	0	0	0
0011	OR	0	0	0	0	0	0
0100	XOR	0	0	0	0	0	0
0101	NOT	0	0	0	0	0	0
0110	SHL	0	0	1	0	0	0
0111	SHR	0	0	0	1	0	0
1000	MUL	0	0	0	0	0	1
1001	DIV	0	0	0	0	1	1
1010 - 1111	unused	0	0	0	0	0	0

Table 9. control_unit - opcode decoding, simulated for all opcodes

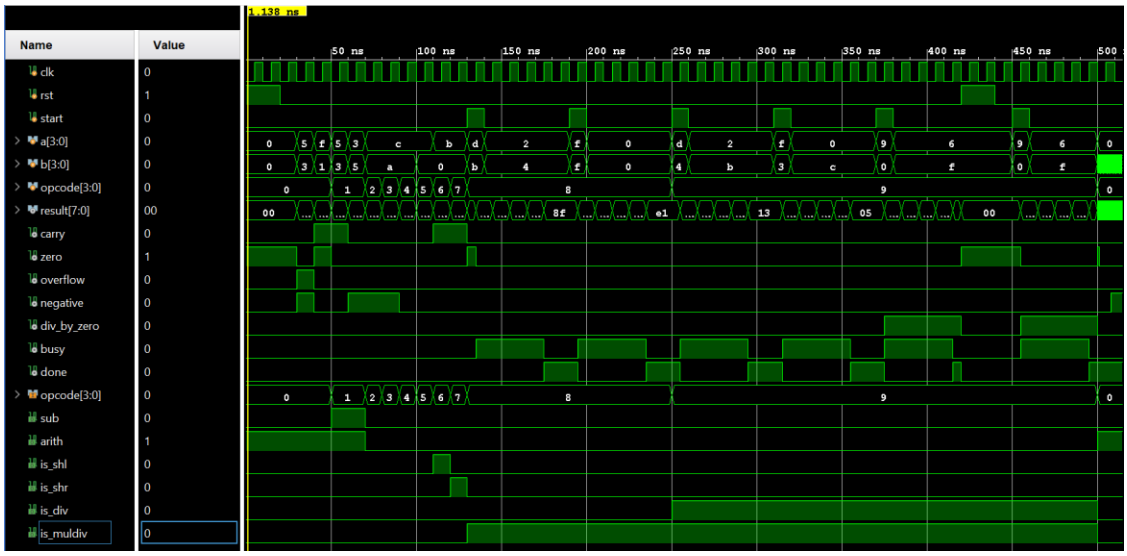


Figure 4. Waveform of control_unit: the decoded control signals follow the opcode.

```
`timescale 1ns / 1ps
// ============================================================
// control_unit.v
// Decodes the 4-bit opcode into the control signals used by
// the rest of the ALU.
//
//   opcode | operation | type
//   -------+-----------+--------------------------
//    0000  | ADD       | single cycle
//    0001  | SUB       | single cycle
//    0010  | AND       | single cycle
//    0011  | OR        | single cycle
//    0100  | XOR       | single cycle
//    0101  | NOT A     | single cycle
//    0110  | SHL       | single cycle
//    0111  | SHR       | single cycle
//    1000  | MUL       | multi-cycle (shift-and-add)
//    1001  | DIV       | multi-cycle (shift-and-subtract)
//    1010-1111 | unused -> result 0
// ============================================================
module control_unit (
    input  wire [3:0] opcode,
    output wire       sub,       // 1 for SUB            (0001)
    output wire       arith,     // 1 for ADD or SUB     (000x)
    output wire       is_shl,    // 1 for shift left     (0110)
    output wire       is_shr,    // 1 for shift right    (0111)
    output wire       is_div,    // 1 for divide         (1001)
    output wire       is_muldiv  // 1 for MUL or DIV     (100x)
);
    assign sub       = (opcode == 4'b0001);
    assign arith     = (opcode[3:1] == 3'b000);
    assign is_shl    = (opcode == 4'b0110);
    assign is_shr    = (opcode == 4'b0111);
    assign is_div    = (opcode == 4'b1001);
    assign is_muldiv = (opcode[3:1] == 3'b100);
endmodule
```

Source code: control_unit.v

4.2. full_adder - One-Bit Adder

Function: full_adder is the basic one-bit building block of the arithmetic path. It adds three bits, a, b and a carry-in, and produces a sum bit and a carry-out. Four instances of this module are chained inside adder_subtractor_4bit to build the full 4-bit ripple-carry adder/subtractor.

Port	Direction	Description
a, b, cin	input	the three bits to add
sum	output	a XOR b XOR cin
cout	output	(a AND b) OR ((a XOR b) AND cin)

Table 10. full_adder - port list

a	b	cin	sum	cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Table 11. full_adder - complete truth table (all 8 cases)

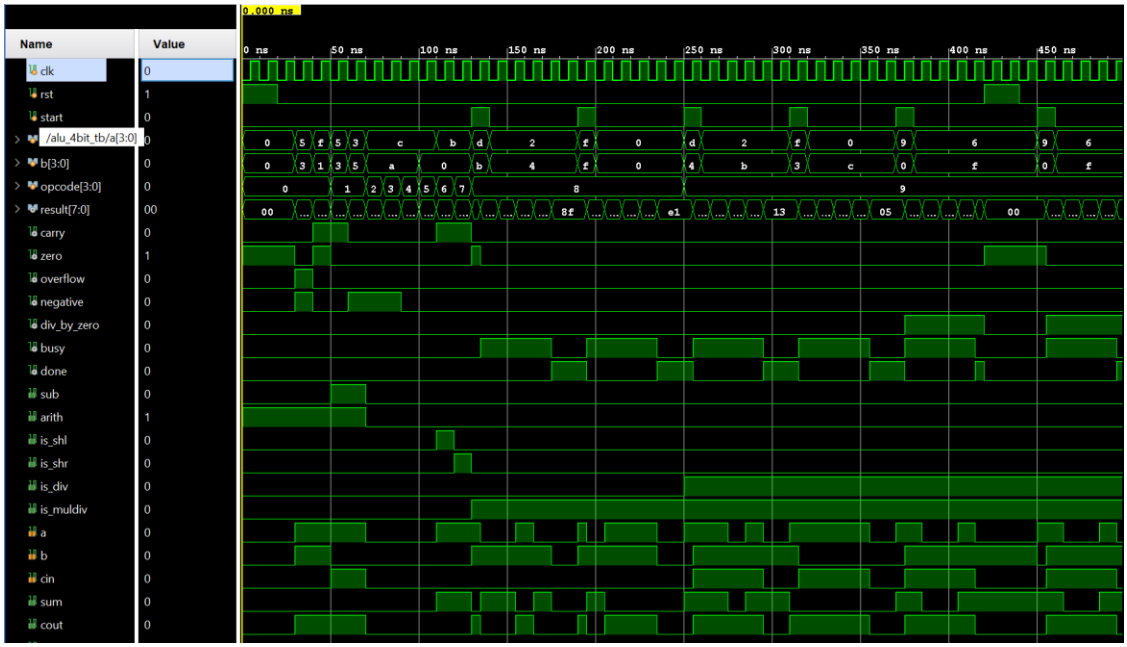


Figure 5. Waveform of full_adder: inputs a, b, cin and outputs sum, cout.

```
`timescale 1ns / 1ps
// ============================================================
// full_adder.v
// 1-bit full adder built from basic gates.
//   sum  = a XOR b XOR cin
//   cout = (a AND b) OR ((a XOR b) AND cin)
// ============================================================
module full_adder (
    input  wire a,
    input  wire b,
    input  wire cin,
    output wire sum,
    output wire cout
);
    wire p;    // propagate = a xor b
```

```
    wire g;    // generate  = a and b
    wire pc;   // p and cin


    xor x1 (p,    a, b);
    xor x2 (sum,  p, cin);
    and a1 (g,    a, b);
    and a2 (pc,   p, cin);
    or  o1 (cout, g, pc);
endmodule
```

Source code: full_adder.v

4.3. adder_subtractor_4bit - Shared Adder / Subtractor

Function: Chains four full_adder instances into a 4-bit ripple-carry adder/subtractor. Each bit of b passes through an XOR gate controlled by sub, and sub also feeds the carry-in of bit 0, so the same chain computes a+b when sub=0 and a-b (two's-complement) when sub=1. It exposes c3 (carry into the sign bit) and c4 (carry out of the sign bit) for flag_unit, and this is the exact adder that muldiv_unit borrows while busy=1.

Port	Direction	Description
a, b[3:0]	input	operands (muxed with muldiv_unit while busy)
sub	input	0 = add, 1 = subtract
sum[3:0]	output	result
c3, c4	output	carry into / out of the MSB

Table 12. adder_subtractor_4bit - port list

Case	sub	a	b	b_x	c1	c2	c3	c4	sum	C	V
5 + 3	0	0101	0011	0011	1	1	1	0	1000 (8)	0	1
15 + 1	0	1111	0001	0001	1	1	1	1	0000 (0)	1	0
8 + 8	0	1000	1000	1000	0	0	0	1	0000 (0)	1	1
5 - 3	1	0101	0011	1100	1	0	1	1	0010 (2)	1	0
3 - 5	1	0011	0101	1010	1	1	0	0	1110 (14)	0	0

Table 13. adder_subtractor_4bit - simulated cases with the full carry chain

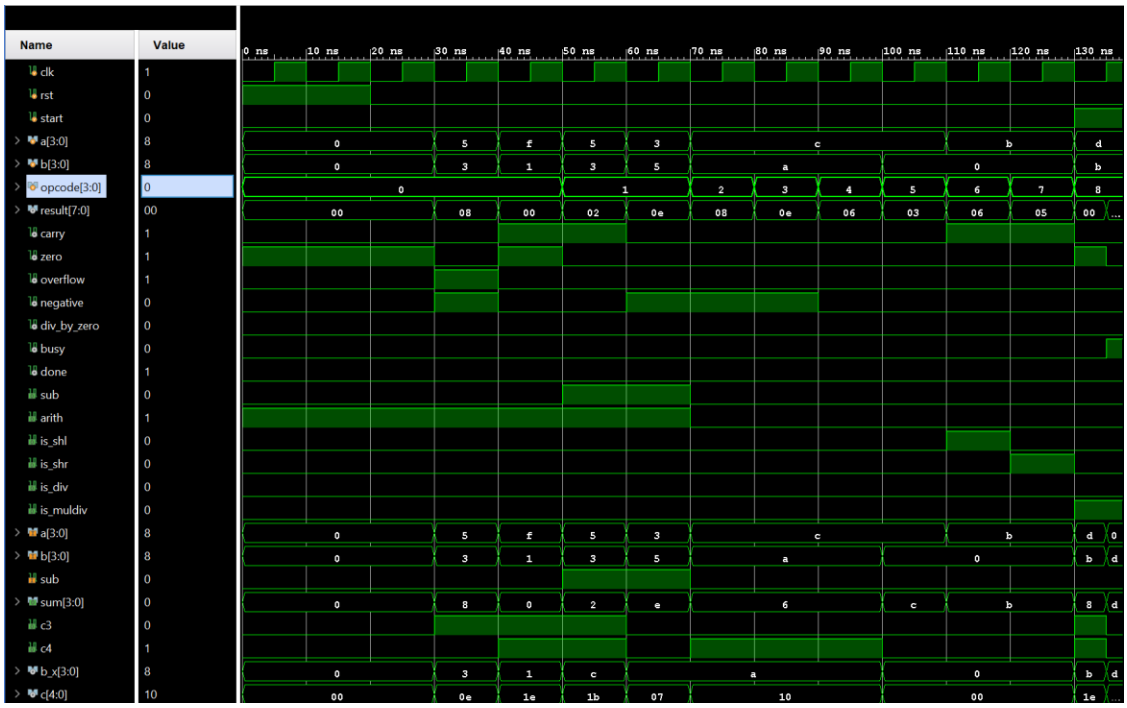


Figure 6. Waveform of adder_subtractor_4bit: sum, c3, c4, b_x and the carry chain c[4:0].

```
`timescale 1ns / 1ps
// ============================================================
// adder_subtractor_4bit.v
// 4-bit ripple-carry adder / subtractor.
//   sub = 0 : sum = a + b
//   sub = 1 : sum = a - b   (computed as a + ~b + 1, two's complement)
// Each bit of b is XORed with 'sub' (inverts b when subtracting),
// and 'sub' is also the carry-in to bit 0 (the "+1").
// Outputs c3 (carry into MSB) and c4 (carry out) for the flag unit.
// ============================================================
module adder_subtractor_4bit (
    input  wire [3:0] a,
    input  wire [3:0] b,
    input  wire       sub,
    output wire [3:0] sum,
    output wire       c3,     // carry into bit 3
    output wire       c4      // carry out of bit 3
);
    wire [3:0] b_x;           // b after optional inversion
    wire [4:0] c;             // internal carry chain

    assign c[0] = sub;

    // invert b when subtracting
    xor xb0 (b_x[0], b[0], sub);
    xor xb1 (b_x[1], b[1], sub);
    xor xb2 (b_x[2], b[2], sub);
    xor xb3 (b_x[3], b[3], sub);

    // ripple chain of four full adders
    full_adder fa0 (.a(a[0]), .b(b_x[0]), .cin(c[0]), .sum(sum[0]), .cout(c[1]));
    full_adder fa1 (.a(a[1]), .b(b_x[1]), .cin(c[1]), .sum(sum[1]), .cout(c[2]));
    full_adder fa2 (.a(a[2]), .b(b_x[2]), .cin(c[2]), .sum(sum[2]), .cout(c[3]));
    full_adder fa3 (.a(a[3]), .b(b_x[3]), .cin(c[3]), .sum(sum[3]), .cout(c[4]));

    assign c3 = c[3];
    assign c4 = c[4];
endmodule
```

Source code: adder_subtractor_4bit.v

4.4. logic_unit_4bit - Bitwise Operations

Function: Computes AND, OR, XOR and NOT on the two 4-bit operands every clock, all four in parallel; result_mux later selects whichever one the opcode asked for. Each output bit depends only on the same bit position of a and (where applicable) b.

Port	Direction	Description
a, b[3:0]	input	operands
and_out, or_out, xor_out, not_out[3:0]	output	the four bitwise results, computed simultaneously

Table 14. logic_unit_4bit - port list

a bit	b bit	AND	OR	XOR	NOT a
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

Table 15. logic_unit_4bit - per-bit truth table

a	b	and_out	or_out	xor_out	not_out
1100 (12)	1010 (10)	1000	1110	0110	0011

Table 16. logic_unit_4bit - simulated 4-bit example

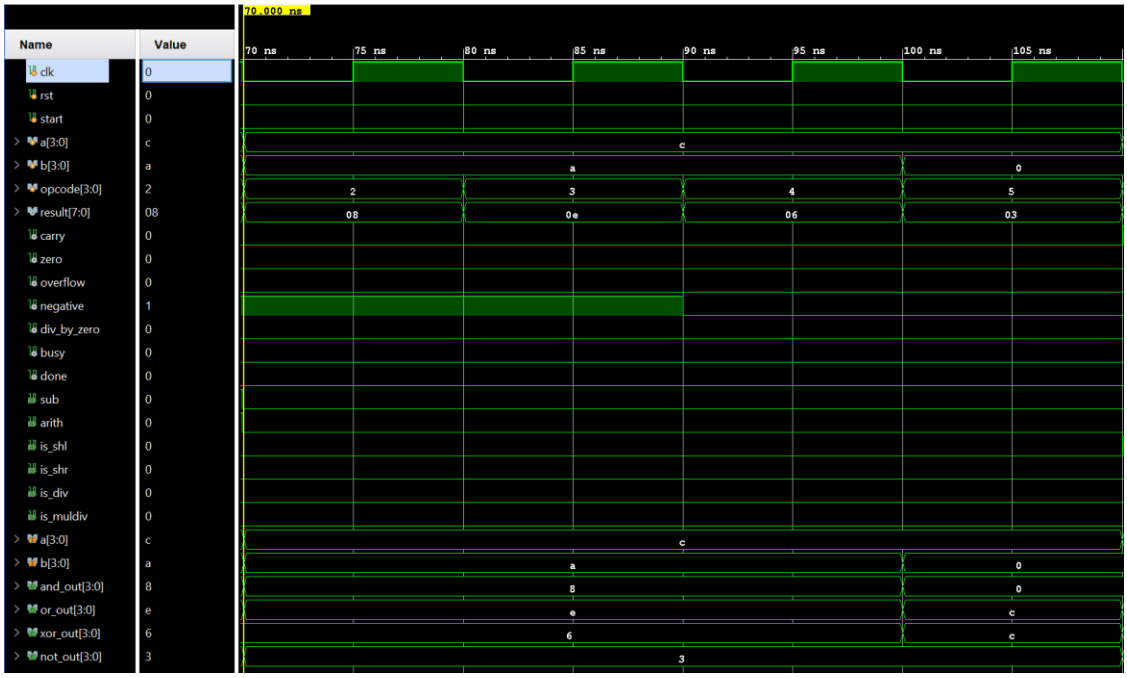


Figure 7. Waveform of logic_unit_4bit: and_out, or_out, xor_out, not_out for a = 1100 and b = 1010.

```
`timescale 1ns / 1ps
// ============================================================
// logic_unit_4bit.v
// Bitwise logic operations, all computed in parallel.
// The result multiplexer picks the one the opcode asks for.
// ============================================================
```

```
module logic_unit_4bit (
    input  wire [3:0] a,
    input  wire [3:0] b,
    output wire [3:0] and_out,   // a & b
    output wire [3:0] or_out,    // a | b
    output wire [3:0] xor_out,   // a ^ b
    output wire [3:0] not_out    // ~a
);
    assign and_out = a & b;
    assign or_out  = a | b;
    assign xor_out = a ^ b;
    assign not_out = ~a;
endmodule
```

Source code: logic_unit_4bit.v

4.5. shifter_4bit - Logical Shift

Function: Performs a logical shift of operand a by exactly one bit position in each direction; SHL and SHR are single-operand instructions, so operand b is not used here. A shift needs no gates, only wiring: shl concatenates the lower three bits of a with a zero, and shr concatenates a zero with the upper three bits of a. The bit that falls off becomes shl_out_bit or shr_out_bit, which flag_unit reports as the carry flag for SHL and SHR.

Port	Direction	Description
a[3:0]	input	operand to shift (b is not used)
shl, shr[3:0]	output	a shifted left / right by 1
shl_out_bit, shr_out_bit	output	the bit shifted out (feeds carry)

Table 17. shifter_4bit - port list

a (dec)	a	shl	shl_out_bit	shr	shr_out_bit
0	0000	0000	0	0000	0
1	0001	0010	0	0000	1
2	0010	0100	0	0001	0
3	0011	0110	0	0001	1
4	0100	1000	0	0010	0
5	0101	1010	0	0010	1
6	0110	1100	0	0011	0
7	0111	1110	0	0011	1
8	1000	0000	1	0100	0
9	1001	0010	1	0100	1
10	1010	0100	1	0101	0
11	1011	0110	1	0101	1
12	1100	1000	1	0110	0
13	1101	1010	1	0110	1
14	1110	1100	1	0111	0
15	1111	1110	1	0111	1

Table 18. shifter_4bit - complete truth table (all 16 values of a)

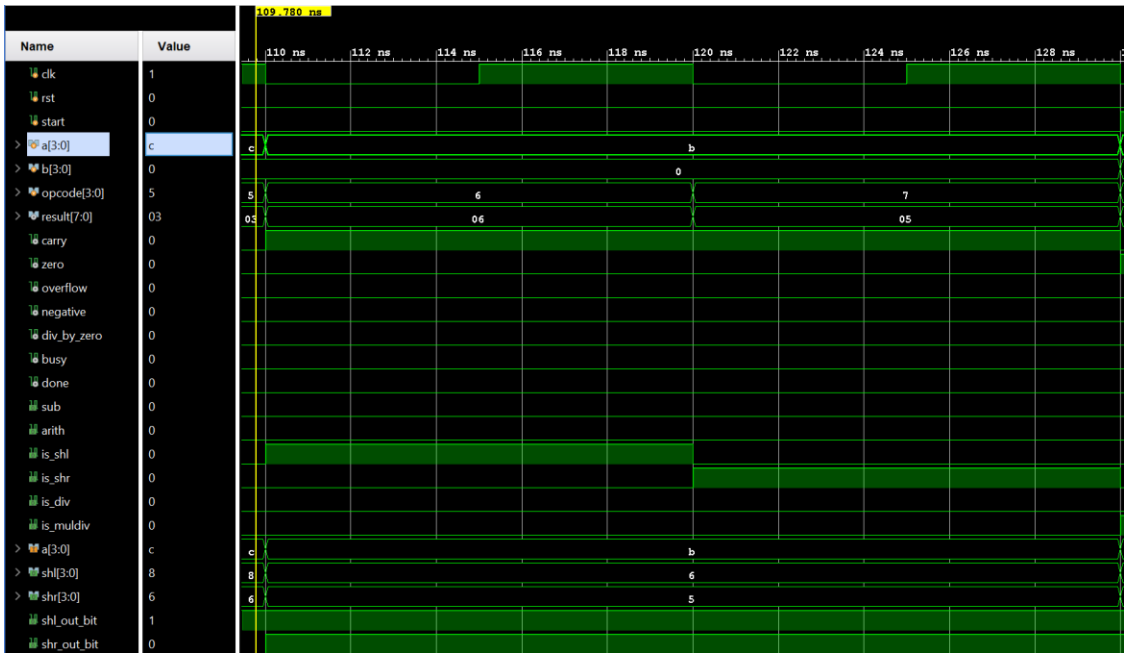


Figure 8. Waveform of shifter_4bit: shl, shr and the bits shifted out, for a = 1100 and a = 1011.

```
`timescale 1ns / 1ps
// ============================================================
// shifter_4bit.v
// Logical shift by one position (pure wiring, no gates needed).
//   Shift left : every bit moves one place left,  0 enters at bit 0,
//                a[3] falls off  -> shl_out_bit   (a * 2)
//   Shift right: every bit moves one place right, 0 enters at bit 3,
//                a[0] falls off  -> shr_out_bit   (a / 2)
// Example a = 1011 : shl = 0110 (bit out 1), shr = 0101 (bit out 1)
// ============================================================
module shifter_4bit (
    input  wire [3:0] a,
    output wire [3:0] shl,
    output wire [3:0] shr,
    output wire       shl_out_bit,
    output wire       shr_out_bit
);
    assign shl         = {a[2:0], 1'b0};   // join lower 3 bits with a 0
    assign shr         = {1'b0, a[3:1]};   // join a 0 with upper 3 bits
    assign shl_out_bit = a[3];
    assign shr_out_bit = a[0];
endmodule
```

Source code: shifter_4bit.v

4.6. muldiv_unit - Sequential Multiplier / Divider (Focus Module)

Function: The only clocked module in the design. It holds the three working registers acc, q and m, a 2-bit step counter, and a small run/idle state (running, mode_div, done, div_by_zero), 18 flip-flops in all. It drives the shared adder through add_a, add_b, add_sub and reads back add_sum, add_c4; this is the hardware-sharing connection from Section 2.4. Its outputs hi and lo feed result_mux; busy controls the top-level operand multiplexers; and busy, done, div_by_zero leave the ALU as status outputs.

Port	Direction	Description
clk, rst	input	clock, asynchronous active-high reset
start	input	1-clock pulse to begin MUL or DIV
is_div	input	0 = multiply, 1 = divide
a, b[3:0]	input	operands, latched at start
add_a, add_b, add_sub	output	drives the shared adder
add_sum[3:0], add_c4	input	reads the shared adder's result
hi, lo[3:0]	output	product high/low, or {remainder, quotient}
busy	output	1 while a multi-cycle operation runs
done	output	1 when the result is ready; held until the next start
div_by_zero	output	latched when DIV is started with b = 0

Table 19. muldiv_unit - port list

Reading the step tables: in the DIV tables, partial is the shifted accumulator {acc[2:0], q[3]} that the divisor m is compared with (the value after bringing down the next dividend bit). The divisor fits when partial >= m, that is, when partial - m produces no borrow. (In the RTL this decision is also OR-ed with r_top, which is always 0 for a 4-bit dividend; see Section 2.3.)

Clock	Action	acc	q	m	result[7:0]	busy	done
Load	acc=0000, q=b=1011, m=a=1101	0000	1011	1101	0B	1	0
Step 1	q[0]=1: 0000+1101, shift right	0110	1101	1101	6D	1	0
Step 2	q[0]=1: 0110+1101, shift right	1001	1110	1101	9E	1	0
Step 3	q[0]=0: no add, shift right	0100	1111	1101	4F	1	0
Step 4	q[0]=1: 0100+1101, shift right	1000	1111	1101	8F (143)	0	1

Table 20. muldiv_unit - MUL 13 x 11 = 143, simulated step by step

Clock	Action	acc	q	m	result[7:0]	busy	done
Load	acc=0000, q=a=1101, m=b=0100	0000	1101	0100	0D	1	0
Step 1	partial 0001 < 4: no fit, bit 0	0001	1010	0100	1A	1	0
Step 2	partial 0011 < 4: no fit, bit 0	0011	0100	0100	34	1	0
Step 3	partial 0110 >= 4: fits, bit 1	0010	1001	0100	29	1	0
Step 4	partial 0101 >= 4: fits, bit 1	0001	0011	0100	13 (q=3, r=1)	0	1

Table 21. muldiv_unit - DIV 13 / 4 = quotient 3 remainder 1, simulated step by step

Clock	Action	acc	q	m	result[7:0]	busy	done	dbz
Load	acc=0000, q=a=1001, m=b=0000	0000	1001	0000	09	1	0	1
Step 1	partial 0001 >= 0: fits, bit 1	0001	0011	0000	13	1	0	1
Step 2	partial 0010 >= 0: fits, bit 1	0010	0111	0000	27	1	0	1
Step 3	partial 0100 >= 0: fits, bit 1	0100	1111	0000	4F	1	0	1
Step 4	partial 1001 >= 0: fits, bit 1	1001	1111	0000	9F (q=15, r=9)	0	1	1

Table 22. muldiv_unit - DIV 9 / 0: quotient forced to 1111, remainder = dividend, div_by_zero = 1

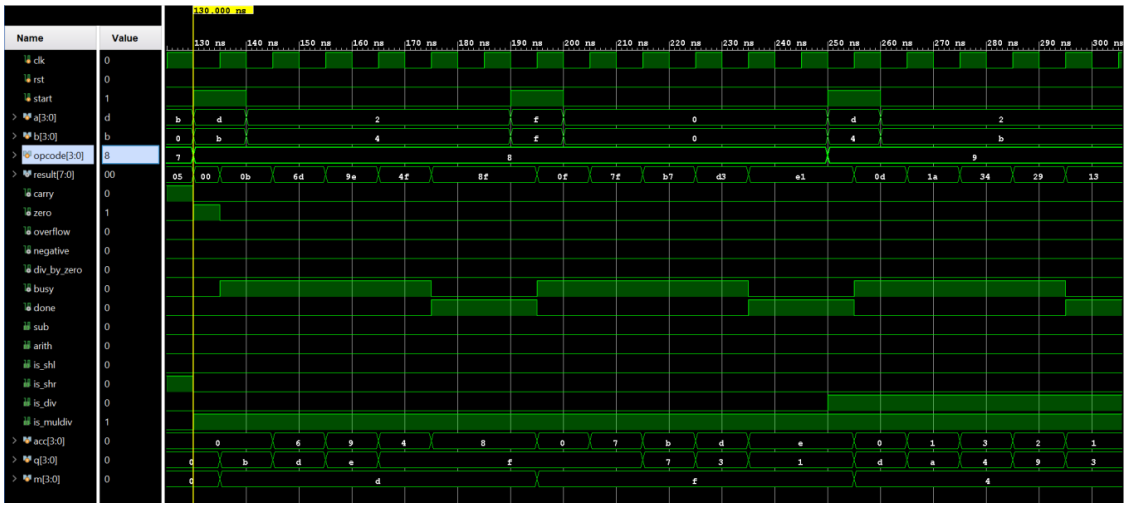


Figure 9. Waveform of muldiv_unit (1 of 3): MUL runs with acc, q and m visible.

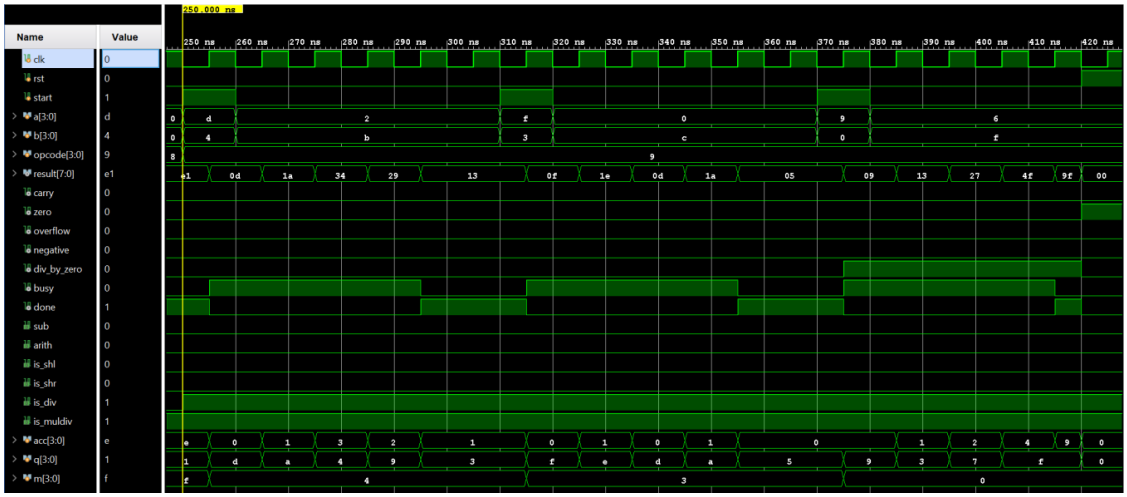


Figure 10. Waveform of muldiv_unit (2 of 3): MUL and DIV runs.

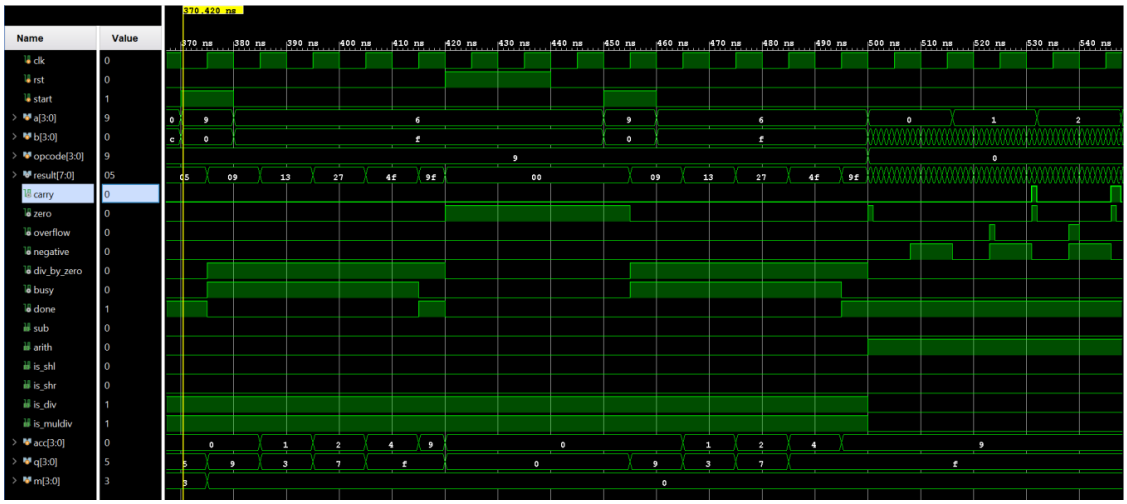


Figure 11. Waveform of muldiv_unit (3 of 3): DIV by zero with div_by_zero set.

```
`timescale 1ns / 1ps
// ============================================================
// muldiv_unit.v
```

```
// Sequential multiplier and divider built from SHIFTING plus
// ONE 4-bit adder/subtractor. The adder is not inside this
// module: it is the ALU's own adder, borrowed while busy = 1.
// So MUL and DIV add only registers and a few muxes to the ALU.
//
// Registers
//   acc : MUL -> upper half of product   | DIV -> remainder
//   q   : MUL -> multiplier, becomes the lower half of product
//         DIV -> dividend, becomes the quotient
//   m   : MUL -> multiplicand            | DIV -> divisor
//
// MULTIPLY (shift-and-add), 4 steps, one per clock:
//   if q[0] == 1 : acc = acc + m      (add multiplicand)
//   shift {carry, acc, q} right by 1
//   after 4 steps: product = {acc, q}
//
// DIVIDE (restoring, shift-and-subtract), 4 steps:
//   shift {acc, q} left by 1          (bring down next dividend bit)
//   try acc - m
//   if it fits (no borrow): acc = acc - m, q[0] = 1
//   else                  : keep acc,      q[0] = 0
//   after 4 steps: quotient = q, remainder = acc
//
// Protocol: put a, b on the inputs, pulse start for one clock.
// busy goes high for 4 clocks, then done goes high and the
// result is held until the next start.
// Divide by zero: div_by_zero = 1, quotient = 1111, remainder = a.
// ============================================================
module muldiv_unit (
    input  wire       clk,
    input  wire       rst,        // active-high asynchronous reset
    input  wire       start,      // pulse for one clock to begin
    input  wire       is_div,     // 0 = multiply, 1 = divide
    input  wire [3:0] a,
    input  wire [3:0] b,

    // connection to the shared adder/subtractor
    output wire [3:0] add_a,
    output wire [3:0] add_b,
    output wire       add_sub,
    input  wire [3:0] add_sum,
    input  wire       add_c4,     // carry out (for SUB: 1 = no borrow)

    output wire [3:0] hi,         // MUL: product[7:4]  DIV: remainder
    output wire [3:0] lo,         // MUL: product[3:0]  DIV: quotient
    output wire       busy,
    output reg        done,
    output reg        div_by_zero
);
    reg [3:0] acc, q, m;
    reg [1:0] count;              // counts the 4 steps (0..3)
    reg       running;
    reg       mode_div;           // operation latched at start

    // divide: {acc, q} shifted left by one
    wire [3:0] r_shift = {acc[2:0], q[3]};
    wire       r_top   = acc[3];            // bit shifted out of acc
    wire       fits    = r_top | add_c4;    // remainder >= divisor ?

    // drive the shared adder
    assign add_a   = mode_div ? r_shift : acc;
    assign add_b   = m;
    assign add_sub = mode_div;              // subtract when dividing

    assign busy = running;
    assign hi   = acc;
    assign lo   = q;

    always @(posedge clk or posedge rst) begin
        if (rst) begin
            acc <= 4'd0;  q <= 4'd0;  m <= 4'd0;
            count <= 2'd0;  running <= 1'b0;  mode_div <= 1'b0;
            done <= 1'b0;  div_by_zero <= 1'b0;
        end
        else if (start && !running) begin
            // load
            running     <= 1'b1;
            done        <= 1'b0;
```

```
            mode_div    <= is_div;
            count       <= 2'd0;
            acc         <= 4'd0;
            q           <= is_div ? a : b;    // dividend  / multiplier
            m           <= is_div ? b : a;    // divisor   / multiplicand
            div_by_zero <= is_div && (b == 4'd0);
        end
        else if (running) begin
            if (!mode_div) begin
                // ---- one multiply step: add (if q[0]) then shift right
                if (q[0])
                    {acc, q} <= {add_c4, add_sum, q[3:1]};
                else
                    {acc, q} <= {1'b0,   acc,     q[3:1]};
            end
            else begin
                // ---- one divide step: shift left, subtract if it fits
                if (fits) begin
                    acc <= add_sum;           // r_shift - m
                    q   <= {q[2:0], 1'b1};
                end
                else begin
                    acc <= r_shift;           // restore (keep)
                    q   <= {q[2:0], 1'b0};
                end
            end

            count <= count + 2'd1;
            if (count == 2'd3) begin
                running <= 1'b0;
                done    <= 1'b1;
            end
        end
    end
endmodule
```

Source code: muldiv_unit.v

4.7. result_mux - Final Result Selection

Function: A combinational multiplexer that chooses which unit's output becomes the 8-bit ALU result, based purely on the opcode. Single-cycle results are zero-padded to 8 bits; MUL and DIV pass the full {hi, lo} pair through unchanged. Any undefined opcode produces a constant zero.

Port	Direction	Description
opcode[3:0]	input	selects the source
sum, and_out, or_out, xor_out, not_out, shl, shr[3:0]	input	single-cycle results
md_hi, md_lo[3:0]	input	MUL/DIV result
result[7:0]	output	the selected 8-bit answer

Table 23. result_mux - port list

opcode	Operation	Selected input
0000	ADD	sum
0001	SUB	sum
0010	AND	and_out
0011	OR	or_out
0100	XOR	xor_out
0101	NOT	not_out
0110	SHL	shl
0111	SHR	shr
1000	MUL	{md_hi, md_lo}
1001	DIV	{md_hi, md_lo}
1010 - 1111	unused	none (constant 0)

Table 24. result_mux - selection by opcode

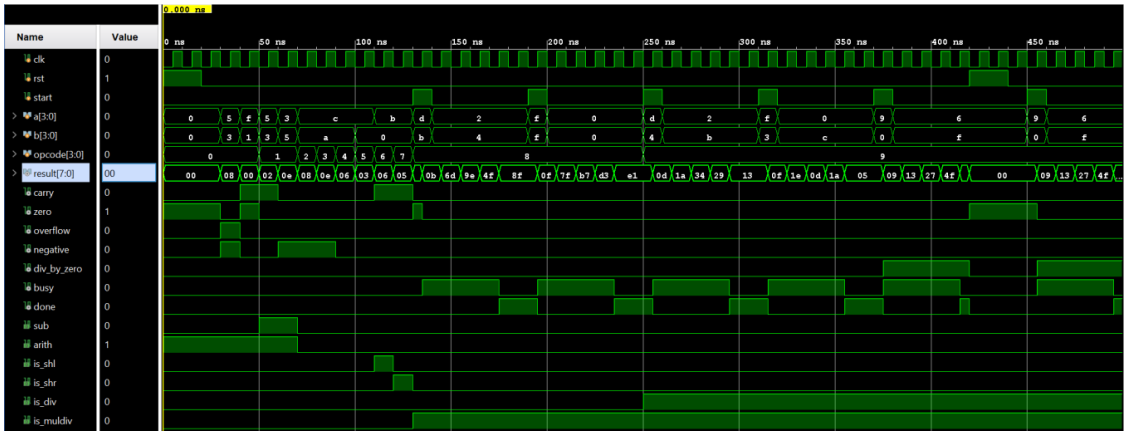


Figure 12. Waveform of the ALU result over the whole simulation (selected by result_mux).

```
`timescale 1ns / 1ps
// ============================================================
// result_mux.v
// Chooses which unit's output becomes the ALU result.
// The result is 8 bits wide so it can hold a full product.
//   single-cycle ops : result = {0000, 4-bit answer}
//   MUL              : result = {product high, product low}
//   DIV              : result = {remainder, quotient}
// ============================================================
module result_mux (
    input  wire [3:0] opcode,
    input  wire [3:0] sum,
    input  wire [3:0] and_out, or_out, xor_out, not_out,
    input  wire [3:0] shl, shr,
    input  wire [3:0] md_hi, md_lo,
    output reg  [7:0] result
);
    always @(*) begin
        case (opcode)
            4'b0000, 4'b0001: result = {4'b0000, sum};
            4'b0010:          result = {4'b0000, and_out};
            4'b0011:          result = {4'b0000, or_out};
            4'b0100:          result = {4'b0000, xor_out};
            4'b0101:          result = {4'b0000, not_out};
            4'b0110:          result = {4'b0000, shl};
            4'b0111:          result = {4'b0000, shr};
            4'b1000, 4'b1001: result = {md_hi, md_lo};
            default:          result = 8'b0000_0000;
        endcase
    end
```

```
endmodule
```

Source code: result_mux.v

4.8. flag_unit - Status Flags

Function: Derives the four arithmetic status flags from the result, the control signals arith / is_shl / is_shr / is_muldiv, the two adder carries c3 and c4, and the two bits shifted out of the shifter. carry = c4 for ADD/SUB, the shifted-out bit for SHL/SHR, 0 otherwise. overflow = c3 XOR c4, meaningful only for ADD/SUB. zero is the NOR of all 8 result bits. negative is result[3], forced to 0 for MUL/DIV since their results are unsigned.

Port	Direction	Description
result[7:0]	input	final ALU result
arith, is_shl, is_shr, is_muldiv	input	which flag rule applies
c3, c4	input	carries from adder_subtractor_4bit
shl_out_bit, shr_out_bit	input	bits shifted out
carry, zero, overflow, negative	output	the four status flags

Table 25. flag_unit - port list

Case	arith	is_shl	is_shr	is_muldiv	c3	c4	C	V	Z	N
ADD 5+3	1	0	0	0	1	0	0	1	0	1
SUB 3-5	1	0	0	0	0	0	0	0	0	1
SHL 11	0	1	0	0	x	x	1	0	0	0
MUL 13x11	0	0	0	1	x	x	0	0	0	0
DIV 9/0	0	0	0	1	x	x	0	0	0	0

Table 26. flag_unit - simulated inputs and resulting flags (representative rows; x = don't care: c3 and c4 do not affect the flags when arith = 0 and depend on the operands present on a and b)

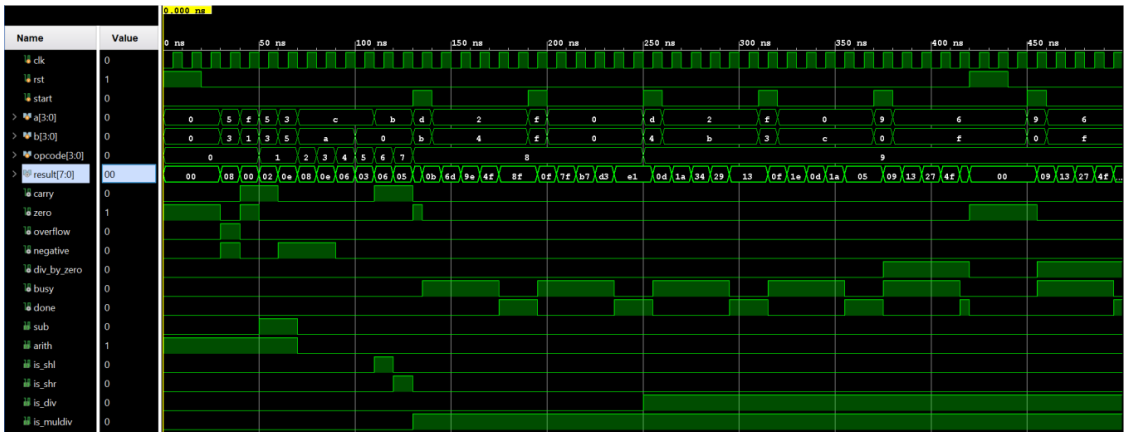


Figure 13. Waveform of the flags carry, zero, overflow and negative over the whole simulation.

```
`timescale 1ns / 1ps
// ============================================================
// flag_unit.v
// Status flags.
//   carry    : ADD -> carry out, SUB -> 1 = no borrow,
//              SHL/SHR -> bit shifted out, others -> 0
//   overflow : signed overflow on ADD/SUB (C3 xor C4), others 0
//   zero     : whole 8-bit result is 0
//   negative : result[3] for 4-bit ops (sign bit);
```

```
//              0 for MUL/DIV (unsigned)
// ============================================================
module flag_unit (
    input  wire [7:0] result,
    input  wire       arith,
    input  wire       is_shl,
    input  wire       is_shr,
    input  wire       is_muldiv,
    input  wire       c3,
    input  wire       c4,
    input  wire       shl_out_bit,
    input  wire       shr_out_bit,
    output wire       carry,
    output wire       zero,
    output wire       overflow,
    output wire       negative
);
    assign carry    = (arith  & c4)
                    | (is_shl & shl_out_bit)
                    | (is_shr & shr_out_bit);
    assign overflow = arith & (c3 ^ c4);
    assign zero     = ~|result;              // NOR of all 8 bits
    assign negative = ~is_muldiv & result[3];
endmodule
```

Source code: flag_unit.v

4.9. alu_4bit - Top Level

Function: Contains no computation of its own beyond the three operand multiplexers that implement adder sharing. Its role is to instantiate the seven sub-modules and connect them: it gates div_by_zero with is_div, and gates start with is_muldiv so that pulsing start while any other opcode is present has no effect.

Port	Direction	Description
clk	input	clock (used only by MUL/DIV)
rst	input	asynchronous reset, active high
start	input	pulse for one clock to begin MUL/DIV
a, b[3:0]	input	operands
opcode[3:0]	input	operation select
result[7:0]	output	answer
carry, zero, overflow, negative	output	status flags
div_by_zero	output	last DIV had b = 0
busy, done	output	MUL/DIV running / finished

Table 27. alu_4bit - port list

Operation	opcode	a	b	result[7:0]	Meaning	C	Z	V	N / Clocks
ADD	0000	5	3	00001000	8; signed overflow	0	0	1	1 / 1
SUB	0001	3	5	00001110	-2 (14 unsigned); borrow	0	0	0	1 / 1
SHL	0110	11	0 (unused)	00000110	1011<<1, bit out 1	1	0	0	0 / 1
MUL	1000	13	11	10001111	product 143	0	0	0	0 / 5
DIV	1001	13	4	00010011	quotient 3, remainder 1	0	0	0	0 / 5
DIV	1001	9	0	10011111	quot 15, rem 9, dbz=1	0	0	0	0 / 5

Table 28. alu_4bit - consolidated operation table, simulated (representative rows; the exhaustive check is in Section 6)

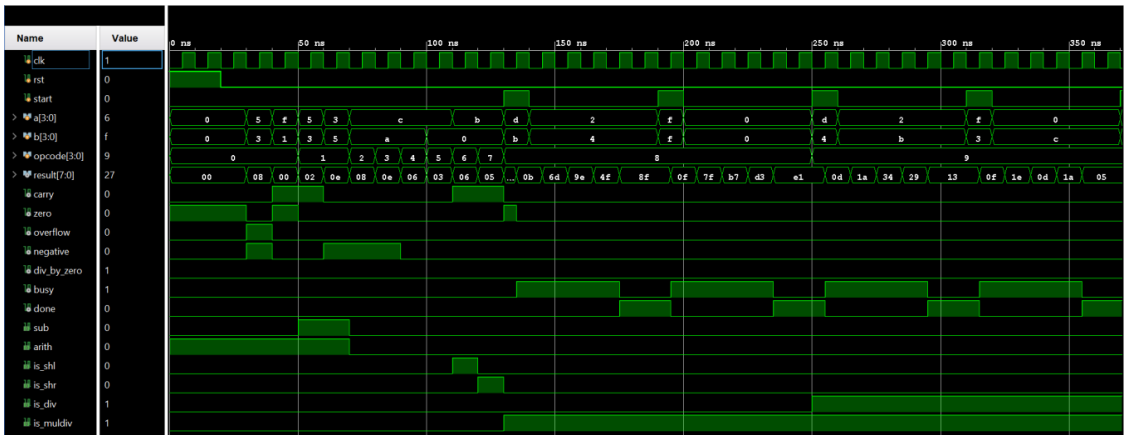


Figure 14. Waveform of alu_4bit: result, flags, busy and done for one example of each operation.

```
`timescale 1ns / 1ps
// ============================================================
// alu_4bit.v  --  TOP LEVEL
// 4-bit ALU with 10 operations. MUL and DIV are done by
// shifting and re-using the ONE adder/subtractor of the ALU,
// instead of building a separate multiplier/divider array.
//
//   opcode: 0000 ADD | 0001 SUB | 0010 AND | 0011 OR
//           0100 XOR | 0101 NOT | 0110 SHL | 0111 SHR
//           1000 MUL | 1001 DIV
//
// Single-cycle ops: result is ready immediately (combinational).
// MUL / DIV       : pulse 'start' for one clock, wait for 'done'
//                   (4 clock cycles of work).
// ============================================================
module alu_4bit (
    input  wire       clk,
    input  wire       rst,
    input  wire       start,
    input  wire [3:0] a,
    input  wire [3:0] b,
    input  wire [3:0] opcode,
    output wire [7:0] result,
    output wire       carry,
    output wire       zero,
    output wire       overflow,
    output wire       negative,
    output wire       div_by_zero,
    output wire       busy,
    output wire       done
);
    // ---------------- control ----------------
    wire sub, arith, is_shl, is_shr, is_div, is_muldiv;

    control_unit u_ctrl (
        .opcode(opcode), .sub(sub), .arith(arith),
        .is_shl(is_shl), .is_shr(is_shr),
        .is_div(is_div), .is_muldiv(is_muldiv)
    );

    // ---------------- shared adder / subtractor ----------------
    // Normally fed from a, b.  While MUL/DIV is busy it is fed
    // by the muldiv unit instead (this is the hardware sharing).
    wire [3:0] md_add_a, md_add_b;
    wire       md_add_sub;
    wire [3:0] add_in_a  = busy ? md_add_a   : a;
    wire [3:0] add_in_b  = busy ? md_add_b   : b;
    wire       add_sub   = busy ? md_add_sub : sub;
    wire [3:0] sum;
    wire       c3, c4;

    adder_subtractor_4bit u_addsub (
        .a(add_in_a), .b(add_in_b), .sub(add_sub),
        .sum(sum), .c3(c3), .c4(c4)
    );
```

```
    // ---------------- logic and shift ----------------
    wire [3:0] and_out, or_out, xor_out, not_out;
    wire [3:0] shl, shr;
    wire       shl_out_bit, shr_out_bit;

    logic_unit_4bit u_logic (
        .a(a), .b(b),
        .and_out(and_out), .or_out(or_out),
        .xor_out(xor_out), .not_out(not_out)
    );

    shifter_4bit u_shift (
        .a(a), .shl(shl), .shr(shr),
        .shl_out_bit(shl_out_bit), .shr_out_bit(shr_out_bit)
    );

    // ---------------- multiply / divide ----------------
    wire [3:0] md_hi, md_lo;
    wire       md_dbz;

    muldiv_unit u_muldiv (
        .clk(clk), .rst(rst),
        .start(start & is_muldiv), .is_div(is_div),
        .a(a), .b(b),
        .add_a(md_add_a), .add_b(md_add_b), .add_sub(md_add_sub),
        .add_sum(sum), .add_c4(c4),
        .hi(md_hi), .lo(md_lo),
        .busy(busy), .done(done), .div_by_zero(md_dbz)
    );

    assign div_by_zero = is_div & md_dbz;

    // ---------------- result select ----------------
    result_mux u_mux (
        .opcode(opcode), .sum(sum),
        .and_out(and_out), .or_out(or_out),
        .xor_out(xor_out), .not_out(not_out),
        .shl(shl), .shr(shr),
        .md_hi(md_hi), .md_lo(md_lo),
        .result(result)
    );

    // ---------------- flags ----------------
    flag_unit u_flags (
        .result(result), .arith(arith),
        .is_shl(is_shl), .is_shr(is_shr), .is_muldiv(is_muldiv),
        .c3(c3), .c4(c4),
        .shl_out_bit(shl_out_bit), .shr_out_bit(shr_out_bit),
        .carry(carry), .zero(zero), .overflow(overflow), .negative(negative)
    );
endmodule
```

Source code: alu_4bit.v

5. Vivado Flow - F1 to F6

5.1. F1 - Sources Window

Shows the complete design hierarchy of alu_4bit, confirming that the project matches the nine modules and the testbench discussed in this report.

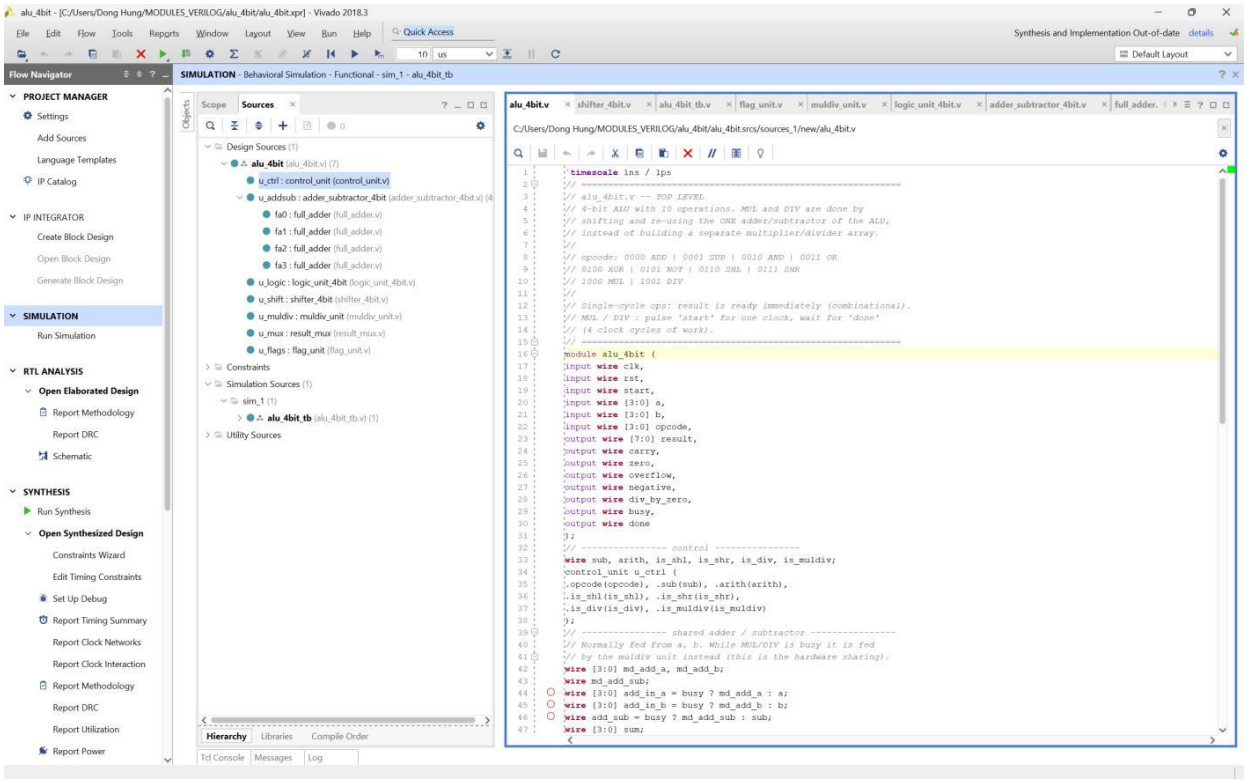


Figure 15. F1 - Sources window, showing the complete design hierarchy of alu_4bit.

5.2. F2 and F3 - Behavioural Simulation

F2 is the waveform window covering the first 340 nanoseconds of simulation (Part 1 of the testbench: one held example per operation). F3 is the console output of the testbench at the end of the run (after run all):

the table of the 15 examples of Part 1, the four [PASS] lines of Part 2, the count of 4,096 / 4,096 input combinations with 30,563 comparisons and zero errors, and the concluding PASS line.

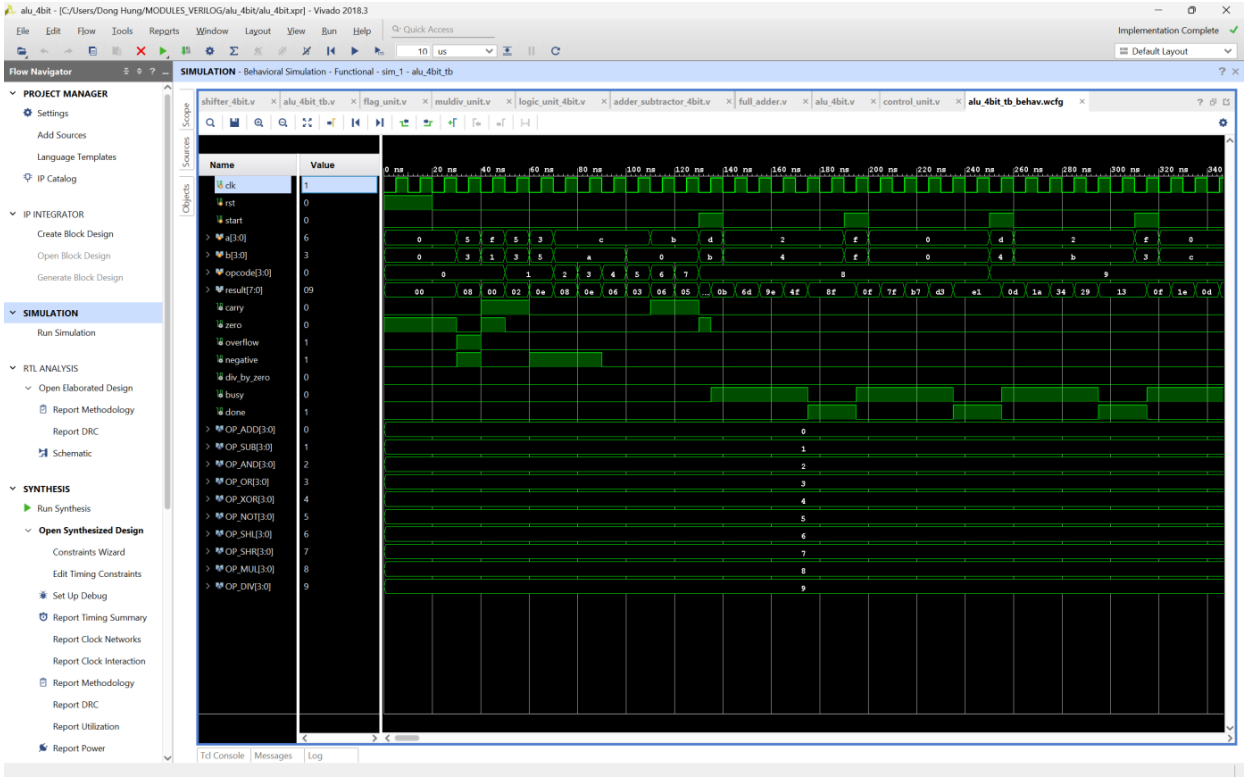


Figure 16. F2 - Behavioural simulation waveform.

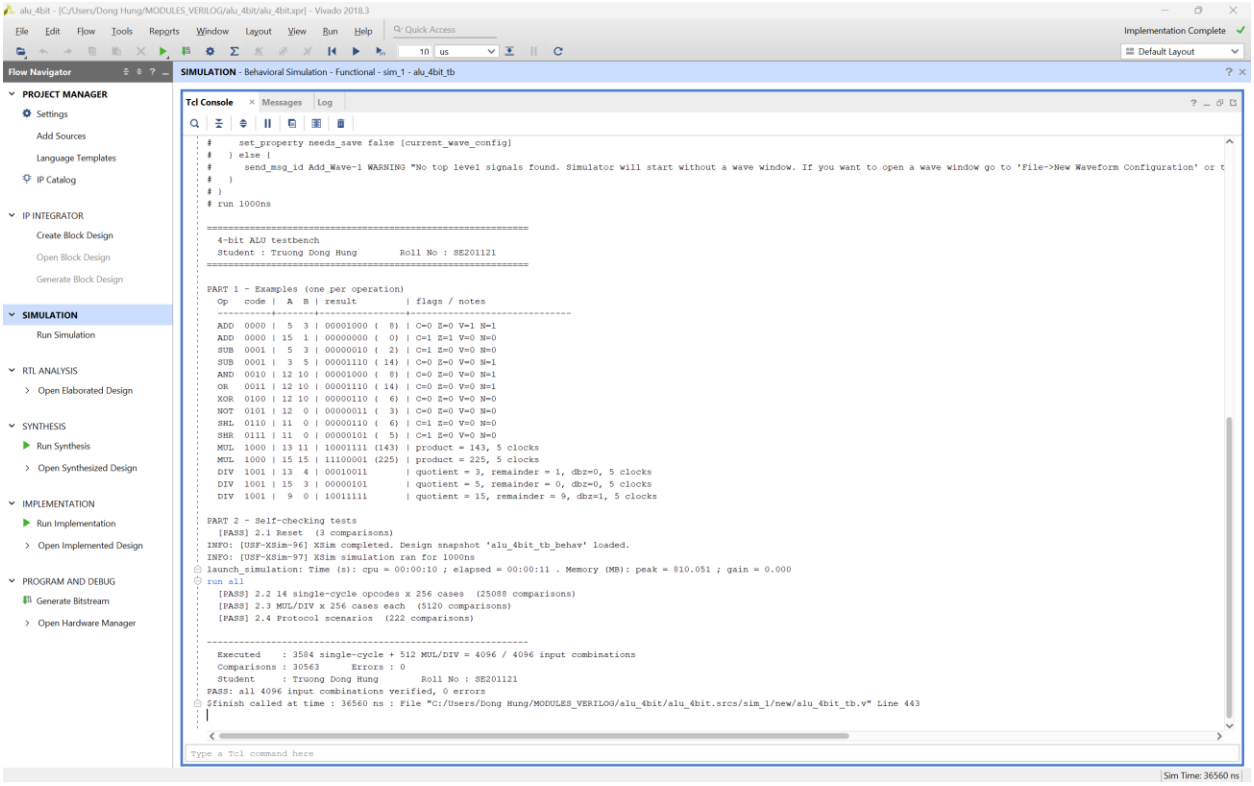


Figure 17. F3 - Tcl Console

5.3. F4 - Elaborated Schematic

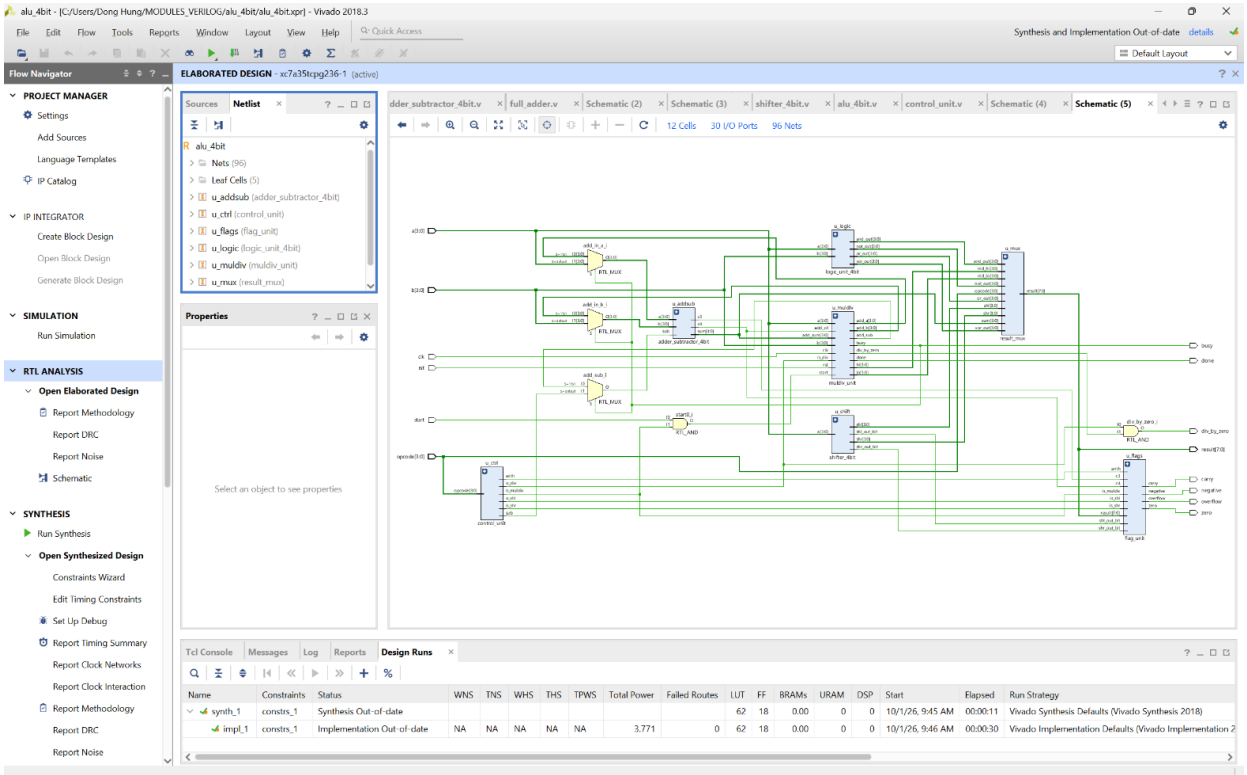


Figure 18. F4 - Elaborated schematic.

5.4. F5 - Synthesised Schematic

The post-synthesis schematic, after the generic RTL has been mapped onto Artix-7 LUTs, buffers and the clock buffer BUFG.

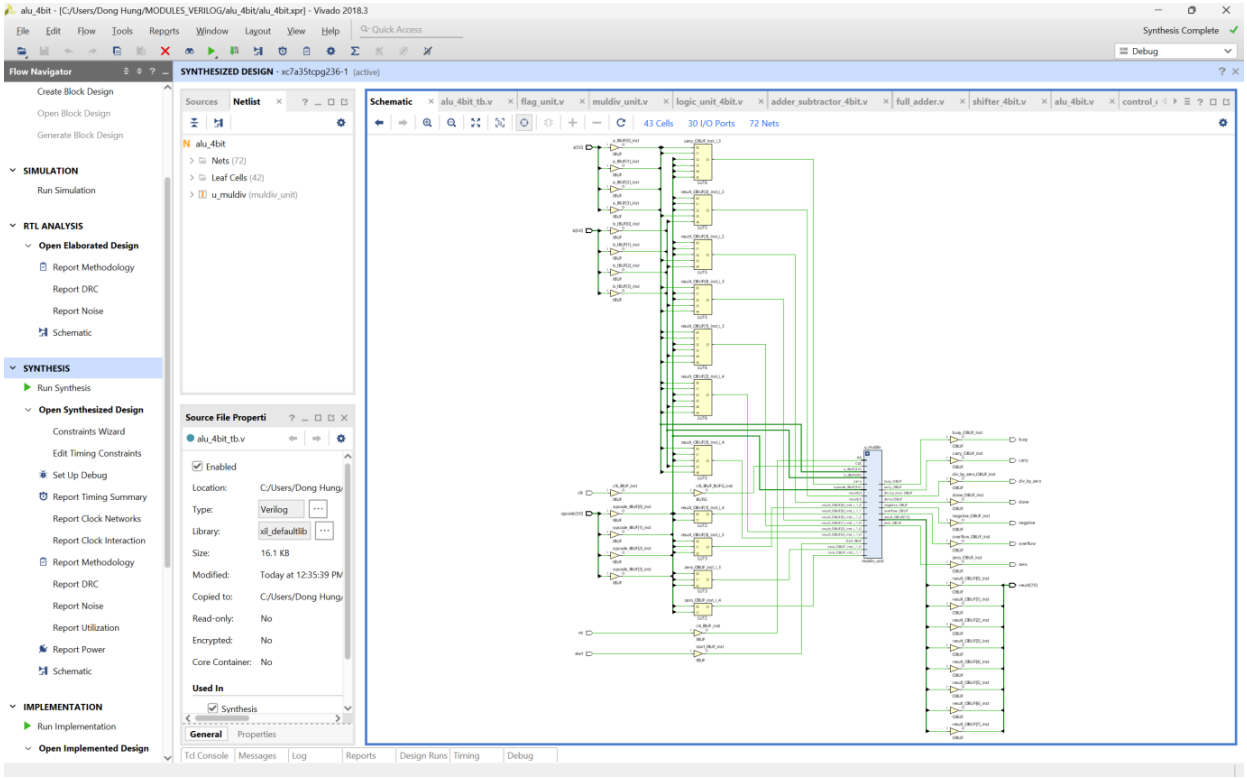


Figure 19. F5 - Synthesised schematic.

5.5. F6 - Utilisation Report

alu_4bit uses **62 Slice LUTs**, **18 Slice Registers** (in **19 Slices**), 1 F7 Mux, 30 Bonded IOB and 1 BUFGCTRL; in the Vivado hierarchy report the sub-hierarchy u_muldiv holds **54 of the 62 LUTs and all**

18 registers, confirming that the entire sequential cost of the design belongs to the multiply/divide unit (the LUT attribution is discussed in Section 7.1).

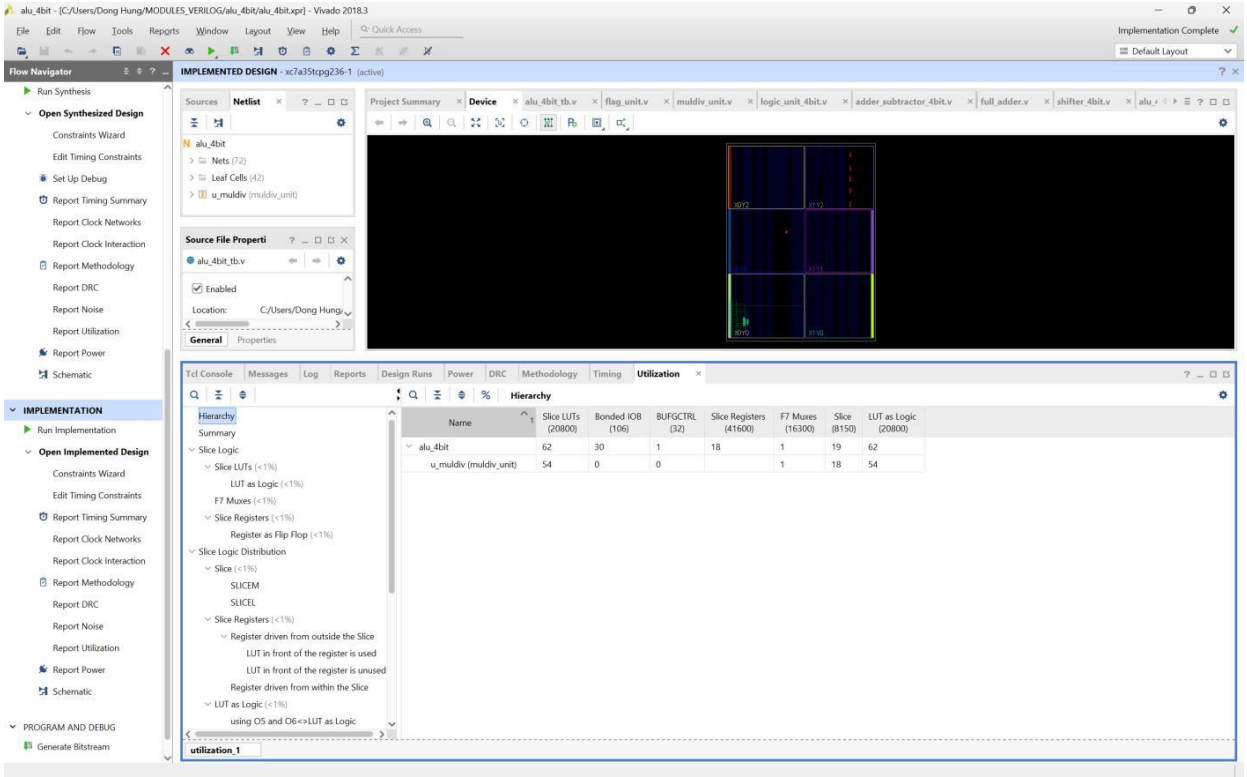


Figure 20. F6 - Utilisation report of the implemented design.

6. Testbench and Verification

6.1. Strategy

alu_4bit_tb.v is self-checking: it computes the expected answer for every stimulus itself, using plain integer arithmetic rather than copying the RTL's own formulas, and compares it against the ALU's actual outputs with Verilog's case-inequality operator !== (which also catches unknown X values). It runs in two parts. Part 1 drives one example of every operation, held for a full clock so it is easy to find on a waveform, and checks it as well. Part 2 then runs four sections: a reset check; all 3,584 combinations of the 14 single-cycle opcodes (the eight operations plus six unused opcodes); all 512 combinations of MUL and DIV, including a 5-clock latency check and an operand-latching check; and a set of protocol scenarios (start while busy, start with a wrong opcode, opcode changed mid-operation, done/result held, div_by_zero gating, and reset applied mid-operation). Together this reaches the full 4,096 input combinations (16 opcodes x 256 operand pairs), with nothing skipped. Every section prints a [PASS] or [FAIL] line, the testbench verifies that every case was really executed, and the last line printed is the verdict: PASS or FAIL.

Operation	Expected value computed by the testbench
ADD	x + y: low 4 bits are the result, carry = (x + y > 15); overflow when the signed sum (operands read as -8..7) is outside -8..7
SUB	x - y: carry = (x >= y), i.e. 1 means no borrow; overflow when the signed difference is outside -8..7
AND, OR, XOR, NOT	x & y, x | y, x ^ y, ~x
SHL, SHR	x << 1 with carry = x[3]; x >> 1 with carry = x[0]
MUL	x * y
DIV	quotient x / y and remainder x % y; for y = 0: quotient 1111, remainder x, div_by_zero = 1
unused opcodes	result = 0
zero, negative	single-cycle opcodes: zero = (4-bit result is 0), negative = result[3]; MUL / DIV: zero = (8-bit result is 0), negative = 0

Table 29. Reference model used by the testbench

6.2. Protocol Scenarios (Section 2.4)

Scenario	What is checked
start while busy	A second start with new operands is ignored: 13 x 3 must give 39, not 15 x 15 = 225
start with a non-MUL / DIV opcode	busy stays 0 for all 14 other opcodes
Opcode changed while running	After switching the opcode to DIV, ADD, SUB and back to MUL during the run, 13 x 11 still gives 143
done and result held	Both stay unchanged for 10 clocks (MUL 12 x 12, DIV 13 / 4); the next start clears done and raises busy
div_by_zero gating	After 9 / 0 the output is 1 with opcode DIV, 0 with ADD, and is cleared by the next MUL
Reset in the middle	An asynchronous reset during a run clears busy and done at once; the ALU works normally afterwards; a late reset clears done and div_by_zero

Table 30. Protocol scenarios checked by the testbench

6.3. Verification Results (Actual Run)

Section	Result	Comparisons	Cases
Part 1: 15 examples, plus the set-up run before 2.2	PASS	130	15
2.1 Reset	PASS	3	-
2.2 14 single-cycle opcodes x 256 cases	PASS	25,088	3,584
2.3 MUL/DIV x 256 cases each	PASS	5,120	512
2.4 Protocol scenarios	PASS	222	-
Total	**PASS, 0 errors**	**30,563**	**4,096 / 4,096**

Table 31. Testbench results (Vivado 2018.3 XSim, simulation finished at 36,560 ns; Icarus Verilog 12 gives the same counts)

The full test needs about 37 microseconds of simulated time. In Vivado the default run time of 1000 ns is too short (the console stops after the [PASS] 2.1 Reset line), so after starting the behavioural simulation the command run all must be typed in the Tcl Console. The last line printed is the verdict: PASS: all 4096 input combinations verified, 0 errors.

6.4. Does the Testbench Catch Bugs?

As a sanity check, 29 deliberate faults were injected into the RTL one at a time and the testbench was re-run. The faults included wrong flag rules, swapped results, a wrong latency, a restoring step that does not restore, operands read live instead of latched, a start accepted while busy, a start not gated by the opcode, a synchronous instead of asynchronous reset, a reset that leaves done or div_by_zero set, and a missing carry-in for SUB.

The testbench reported [FAIL] for 28 of the 29. The 29th, removing the r_top term of the division step, is an equivalent change: r_top is always 0 (Section 2.3), so the modified design behaves identically and no test can detect it. The protocol scenarios (Section 6.2) and the operand-changing check of section 2.3 of the testbench are what catch the faults concerning start, reset and operand latching; a testbench that only

compared the results of the 4,096 combinations would not detect them. This is evidence, not a proof: it covers only the faults that were tried.

The testbench does not check ADD or SUB or the logic and shift results while busy = 1, the intermediate values of result and the flags during a run, or a start held high for several clocks; these behaviours (Section 3.6) were observed in simulation only.

6.5. Complete Testbench Source

The complete source of alu_4bit_tb.v, exactly as stored in the testbench folder of the repository.

```
`timescale 1ns / 1ps
// ============================================================
// alu_4bit_tb.v  --  Self-checking testbench for alu_4bit
//
// Student : Truong Dong Hung        Roll No : SE201121
//
// IDEA: drive the ALU inputs, compute the correct answer inside
// the testbench, compare it with the ALU outputs. Mismatches are
// printed; the LAST line printed is "PASS: ..." or "FAIL: ...".
//
// PART 1 - Examples
//   One readable example for every operation (incl. MUL, DIV and
//   divide by zero). Each example is held for one clock so it is
//   easy to read in the waveform. The results are also checked.
//
// PART 2 - Self-checking tests
//   2.1 Reset
//   2.2 The 14 single-cycle opcodes (ADD..SHR + 6 unused opcodes),
//       each one with all 256 (a,b) pairs      -> 14 x 256 = 3584 cases
//   2.3 MUL and DIV, each one with all 256 (a,b) pairs
//                                              ->  2 x 256 =  512 cases
//       (also checks: latency in clocks, and that changing a/b after
//        'start' does not change the result)
//   2.4 Protocol scenarios: start while busy, start with a wrong
//       opcode, opcode changed while running, done/result held,
//       divide by zero, reset in the middle
//
// Exhaustive: 16 opcodes x 256 (a,b) pairs = 4096 cases, nothing skipped.
//
// HOW TO RUN IN VIVADO: Run Behavioral Simulation, then type
//   run all
// in the Tcl Console (the default run time of 1000 ns is too short).
// ============================================================


// ---- Student information (printed at the start and before the PASS line) ----
`define STUDENT_NAME "Truong Dong Hung"
`define STUDENT_ID   "SE201121"


module alu_4bit_tb;


  // ---------- 1. Signals and DUT ----------
  reg        clk = 0, rst = 1, start = 0;
  reg  [3:0] a = 0, b = 0, opcode = 0;
  wire [7:0] result;
  wire       carry, zero, overflow, negative, div_by_zero, busy, done;


  alu_4bit dut (
    .clk(clk), .rst(rst), .start(start),
    .a(a), .b(b), .opcode(opcode),
    .result(result), .carry(carry), .zero(zero),
    .overflow(overflow), .negative(negative),
    .div_by_zero(div_by_zero), .busy(busy), .done(done)
  );


  always #5 clk = ~clk;                  // 10 ns clock period


  localparam OP_ADD = 4'b0000, OP_SUB = 4'b0001, OP_AND = 4'b0010,
             OP_OR  = 4'b0011, OP_XOR = 4'b0100, OP_NOT = 4'b0101,
             OP_SHL = 4'b0110, OP_SHR = 4'b0111,
             OP_MUL = 4'b1000, OP_DIV = 4'b1001;


  // ---------- 2. Counters and compare helpers ----------
  integer errors = 0;          // total number of errors
  integer checks = 0;          // total number of comparisons
  integer n_comb = 0;          // single-cycle cases executed (must be 3584)
  integer n_md   = 0;          // MUL/DIV cases executed      (must be  512)
  integer sec_errors, sec_checks;
```

```
task expect_bit;                        // compare 1 bit
  input actual, expected;
  input [8*24-1:0] what;
  begin
    checks = checks + 1;
    if (actual !== expected) begin
      errors = errors + 1;
      if (errors <= 10)
        $display("ERROR t=%0t: %0s = %b, expected %b", $time, what, actual, expected);
    end
  end
endtask


task expect byte;                       // compare 8 bits
  input [7:0] actual, expected;
  input [8*24-1:0] what;
  begin
    checks = checks + 1;
    if (actual !== expected) begin
      errors = errors + 1;
      if (errors <= 10)
        $display("ERROR t=%0t: %0s = %b (%0d), expected %b (%0d)  [op=%b a=%0d b=%0d]",
                 $time, what, actual, actual, expected, expected, opcode, a, b);
    end
  end
endtask


task section_begin;
  begin sec_errors = errors; sec_checks = checks; end
endtask


task section_end;
  input [8*44-1:0] name;
  begin
    if (errors == sec_errors) $display("  [PASS] %0s  (%0d comparisons)", name, checks - sec_checks);
    else                      $display("  [FAIL] %0s  (%0d errors)", name, errors - sec_errors);
  end
endtask


// ---------- 3. Expected answer for single-cycle opcodes ----------
reg [7:0] exp res;
reg       exp_c, exp_v, exp_z, exp_n;


task calc_expected;
  input [3:0] op, x, y;
  integer sx, sy, s;
  reg [3:0] r;
  begin
    sx = x[3] ? x - 16 : x;             // signed value, -8..7
    sy = y[3] ? y - 16 : y;
    r = 0; exp c = 0; exp v = 0;
    case (op)
      OP_ADD: begin
        r = x + y;
        exp_c = (x + y > 15);           // carry: sum does not fit in 4 bits
        s = sx + sy;
        exp v = (s > 7) || (s < -8);    // overflow: signed sum outside -8..7
      end
      OP SUB: begin
        r = x - y;
        exp_c = (x >= y);               // carry = 1 means "no borrow"
        s = sx - sy;
        exp_v = (s > 7) || (s < -8);
      end
      OP_AND: r = x & y;
      OP_OR : r = x | y;
      OP_XOR: r = x ^ y;
      OP NOT: r = ~x;
      OP_SHL: begin r = x << 1; exp_c = x[3]; end   // carry = bit shifted out
      OP_SHR: begin r = x >> 1; exp_c = x[0]; end
      default: r = 0;                   // unused opcode -> result 0
    endcase
    exp_res = {4'b0000, r};
    exp z   = (r == 0);
    exp_n   = r[3];
  end
endtask


// Compare every ALU output with the expected answer (inputs already applied)
task compare_comb;
  input [3:0] op, x, y;
  begin
    calc_expected(op, x, y);
    expect byte(result,   exp res, "result");
    expect_bit (carry,    exp_c,   "carry");
    expect_bit (overflow, exp_v,   "overflow");
    expect bit (zero,     exp z,   "zero");
    expect_bit (negative, exp_n,   "negative");
    expect_bit (div_by_zero, 1'b0, "div_by_zero (must be 0)");
```

```
    expect_bit (busy,        1'b0, "busy (must be 0)");
  end
endtask


// Apply (op,a,b) to the ALU, then compare every output
task check_comb;
  input [3:0] op, x, y;
  begin
    opcode = op; a = x; b = y; start = 0;
    #1;                               // let the combinational logic settle
    compare comb(op, x, y);
  end
endtask


// ---------- 4. Expected answer for MUL / DIV ----------
task calc_md;
  input [3:0] op, x, y;
  begin
    if (op == OP_MUL)  exp_res = x * y;
    else if (y == 0)   exp_res = {x, 4'hF};          // divide by 0: quotient=1111, remainder=a
    else begin exp res[7:4] = x % y; exp res[3:0] = x / y; end   // {remainder, quotient}
  end
endtask


// Check the MUL/DIV result (call when done = 1)
task check_md;
  input [3:0] op, x, y;
  begin
    calc_md(op, x, y);
    expect_bit (done,     1'b1,    "done");
    expect_bit (busy,     1'b0,    "busy");
    expect_byte(result,   exp_res, "result MUL/DIV");
    expect_bit (carry,    1'b0,    "carry");
    expect_bit (overflow, 1'b0,    "overflow");
    expect_bit (negative, 1'b0,    "negative");
    expect_bit (zero,     (exp_res == 0), "zero");
    expect_bit (div by zero, (op == OP_DIV) && (y == 0), "div by zero");
  end
endtask


// Wait for done, at most 10 clocks (bounded so the simulation can never hang)
integer cycles;
task wait_done;
  begin
    while (done !== 1'b1 && cycles < 10) begin
      @(negedge clk); cycles = cycles + 1;
    end
  end
endtask


// Run one MUL/DIV: 1-clock start pulse, wait for done, check
task run_muldiv;
  input [3:0] op, x, y;
  begin
    @(negedge clk); opcode = op; a = x; b = y; start = 1;   // clock 0: start
    #1 expect_bit(busy, 1'b0, "busy not yet high");         // busy may only rise AFTER the clock edge
    @(negedge clk); start = 0; a = ~x; b = ~y;   // change a,b: result must NOT change
    cycles = 1;
    wait_done;
    expect_byte(cycles, 5, "clocks start->done");   // 1 load + 4 steps = 5
    check_md(op, x, y);
  end
endtask


// ---------- 4b. Part 1 helpers: print one example ----------
// A single-cycle example is held for one full clock (10 ns) so it is
// easy to see in the waveform. The result is compared as well.
task show_comb;
  input [3:0] op, x, y;
  input [8*4-1:0] name;
  begin
    @(negedge clk); opcode = op; a = x; b = y; start = 0;
    #1 compare_comb(op, x, y);
    #1 $display("  %s %b | %2d %2d | %b (%3d) | C=%b Z=%b V=%b N=%b",
                name, op, x, y, result, result, carry, zero, overflow, negative);
  end
endtask


task show_md;
  input [3:0] op, x, y;
  input [8*4-1:0] name;
  begin
    run_muldiv(op, x, y);              // runs AND checks the operation
    if (op == OP_MUL)
      $display("  %s %b | %2d %2d | %b (%3d) | product = %0d, %0d clocks",
               name, op, x, y, result, result, result, cycles);
    else
      $display("  %s %b | %2d %2d | %b       | quotient = %0d, remainder = %0d, dbz=%b, %0d clocks",
```

```
              name, op, x, y, result, result[3:0], result[7:4], div_by_zero, cycles);
  end
endtask


task part1_examples;
  begin
    $display("  Op   code |  A  B | result         | flags / notes");
    $display("  ----------+-------+----------------+------------------------------");
    show_comb(OP_ADD,  5,  3, "ADD ");
    show_comb(OP_ADD, 15,  1, "ADD ");
    show_comb(OP_SUB,  5,  3, "SUB ");
    show_comb(OP_SUB,  3,  5, "SUB ");
    show_comb(OP_AND, 12, 10, "AND ");
    show_comb(OP_OR,  12, 10, "OR  ");
    show_comb(OP_XOR, 12, 10, "XOR ");
    show_comb(OP_NOT, 12,  0, "NOT ");
    show_comb(OP_SHL, 11,  0, "SHL ");
    show_comb(OP_SHR, 11,  0, "SHR ");
    show_md  (OP_MUL, 13, 11, "MUL ");
    show_md  (OP_MUL, 15, 15, "MUL ");
    show_md  (OP_DIV, 13,  4, "DIV ");
    show_md  (OP_DIV, 15,  3, "DIV ");
    show_md  (OP_DIV,  9,  0, "DIV ");
  end
endtask


// ---------- 5. Test groups ----------
integer i, op_i;


task test_reset;
  begin
    rst = 1; repeat (2) @(negedge clk); rst = 0; #1;
    expect_bit(busy, 0, "busy after reset");
    expect_bit(done, 0, "done after reset");
    expect_bit(div_by_zero, 0, "div_by_zero after reset");
  end
endtask


task test_single_cycle;               // 14 opcodes x 256 (a,b) pairs
  begin
    for (op_i = 0; op_i < 16; op_i = op_i + 1)
      if (op_i != 8 && op_i != 9)      // 8,9 are MUL/DIV, tested separately
        for (i = 0; i < 256; i = i + 1) begin
          check_comb(op_i, i[7:4], i[3:0]);
          n_comb = n_comb + 1;
        end
  end
endtask


task test_muldiv;                     // MUL and DIV, 256 (a,b) pairs each
  begin
    for (i = 0; i < 256; i = i + 1) begin
      run_muldiv(OP_MUL, i[7:4], i[3:0]);
      run_muldiv(OP_DIV, i[7:4], i[3:0]);   // includes divide by zero
      n_md = n_md + 2;
    end
  end
endtask


task test_start_while_busy;           // a 2nd start while running must be ignored
  begin
    @(negedge clk); opcode = OP_MUL; a = 13; b = 3; start = 1;   // real start: 13*3
    @(negedge clk); a = 15; b = 15; start = 1;                   // fake start while busy
    @(negedge clk); start = 0;
    cycles = 3; wait_done;
    check_md(OP_MUL, 13, 3);          // must be 39, not 15*15 = 225
  end
endtask


task test_start_wrong_opcode;         // start with an opcode that is not MUL/DIV
  begin
    for (op_i = 0; op_i < 16; op_i = op_i + 1)
      if (op_i != 8 && op_i != 9) begin
        @(negedge clk); opcode = op_i; a = 9; b = 7; start = 1;
        #1 expect_bit(busy, 0, "busy right after start");
        @(negedge clk); start = 0;
        expect_bit(busy, 0, "busy after 1 clock");
        repeat (4) begin @(negedge clk); expect_bit(busy, 0, "busy"); end
      end
  end
endtask


task test opcode change;              // change the opcode while the operation runs
  begin
    @(negedge clk); opcode = OP_MUL; a = 13; b = 11; start = 1;  // clock 0
    @(negedge clk); start = 0; opcode = OP DIV;                  // clock 1
    @(negedge clk); opcode = OP_ADD;                             // clock 2
```

```
    @(negedge clk); opcode = OP_SUB;                              // clock 3
    @(negedge clk); opcode = OP_MUL;                              // clock 4: back to MUL
    @(negedge clk);                                               // clock 5: done
    check_md(OP_MUL, 13, 11);          // must still be 143
  end
endtask


task test_done_hold;                   // done and result stay until the next start
  input [3:0] op, x, y;
  begin
    run_muldiv(op, x, y);
    calc_md(op, x, y);
    repeat (10) begin
      @(negedge clk);
      expect_bit (done, 1, "done held");
      expect_byte(result, exp_res, "result held");
    end
    @(negedge clk); start = 1; a = 2; b = 2;                      // new start
    @(negedge clk); start = 0;
    expect_bit(done, 0, "done cleared on start");
    expect_bit(busy, 1, "busy on new start");
    cycles = 1; wait_done;
  end
endtask


task test_divide_by_zero;              // div_by_zero only matters for the DIV opcode
  begin
    run_muldiv(OP_DIV, 9, 0);          // latch div_by_zero = 1
    @(negedge clk); opcode = OP_ADD;  #1 expect_bit(div_by_zero, 0, "dbz with ADD");
    @(negedge clk); opcode = OP_DIV;  #1 expect_bit(div_by_zero, 1, "dbz with DIV");
    run_muldiv(OP_MUL, 3, 3);          // a new operation clears the flag
    @(negedge clk); opcode = OP_DIV;  #1 expect_bit(div_by_zero, 0, "dbz after MUL");
  end
endtask


task test_reset_midway;                // reset while running / after finishing
  begin
    @(negedge clk); opcode = OP_MUL; a = 9; b = 9; start = 1;     // clock 0
    @(negedge clk); start = 0;                                    // clock 1
    @(negedge clk);                                               // clock 2
    expect_bit(busy, 1, "busy before reset");
    rst = 1; #1;                       // asynchronous reset: takes effect immediately
    expect_bit(busy, 0, "busy after reset");
    expect_bit(done, 0, "done after reset");
    #1 rst = 0;
    run_muldiv(OP_MUL, 15, 15);        // works normally after reset


    run_muldiv(OP_DIV, 9, 0);          // done = 1, div_by_zero = 1
    rst = 1; #1;
    expect_bit(done, 0, "done after late reset");
    expect_bit(div_by_zero, 0, "dbz after reset");
    #1 rst = 0;
    run_muldiv(OP_DIV, 15, 4);
  end
endtask


// ---------- 6. Main program ----------
initial begin
  $display("");
  $display("============================================================");
  $display("  4-bit ALU testbench");
  $display("  Student : %0s        Roll No : %0s", `STUDENT_NAME, `STUDENT_ID);
  $display("============================================================");


  // power-on reset (2 clocks), then release
  rst = 1; repeat (2) @(negedge clk); rst = 0;


  // ---------------- PART 1 ----------------
  $display("\nPART 1 - Examples (one per operation)");
  part1_examples;


  // ---------------- PART 2 ----------------
  $display("\nPART 2 - Self-checking tests");


  section_begin;  test_reset;
  section_end("2.1 Reset");


  run_muldiv(OP_DIV, 9, 0);            // latch div_by_zero=1 to check it does not leak
  section_begin;  test_single_cycle;
  section_end("2.2 14 single-cycle opcodes x 256 cases");


  section_begin;  test_muldiv;
  section_end("2.3 MUL/DIV x 256 cases each");


  section_begin;
```

```
    test_start_while_busy;
    test_start_wrong_opcode;
    test_opcode_change;
    test_done_hold(OP_MUL, 12, 12);
    test_done_hold(OP_DIV, 13, 4);
    test_divide_by_zero;
    test_reset_midway;
    section_end("2.4 Protocol scenarios");


    // make sure every case was really executed
    if (n_comb != 3584 || n_md != 512) begin
      errors = errors + 1;
      $display("ERROR: not all cases executed (%0d/3584, %0d/512)", n_comb, n_md);
    end


    // ---------------- summary: the LAST line is PASS or FAIL ----------------
    $display("\n-------------------------------------------------------------");
    $display("  Executed    : %0d single-cycle + %0d MUL/DIV = %0d / 4096 input combinations",
             n_comb, n_md, n_comb + n_md);
    $display("  Comparisons : %0d      Errors : %0d", checks, errors);
    $display("  Student     : %0s        Roll No : %0s", `STUDENT_NAME, `STUDENT_ID);
    if (errors == 0)
      $display("PASS: all 4096 input combinations verified, 0 errors");
    else
      $display("FAIL: %0d errors found", errors);
    $finish;
  end


endmodule
```

Source code: alu_4bit_tb.v

7. Synthesis, Implementation, Results

7.1. Utilisation and Power

After synthesis and implementation targeting xc7a35tcpg236-1, alu_4bit occupies 62 Slice LUTs out of 20,800 available, 18 Slice Registers out of 41,600 available (packed into 19 Slices), 1 F7 Mux, 30 Bonded IOB pins out of 106, and 1 BUFGCTRL, all under one percent device utilisation except I/O pins (about 28 percent, because every input/output bit is brought to a separate pin in this simulation-oriented project: 15 inputs and 15 outputs). No .xdc timing constraint was supplied, so timing slack is not applicable, and the default power estimate that Vivado gives for an unconstrained design (3.771 W) is not meaningful for this ALU; neither figure is used in this report.

Resource (xc7a35tcpg236-1)	alu_4bit (total)	of which u_muldiv	Available
Slice LUTs	62	54	20,800
Slice Registers	18	18	41,600
Slices	19	18	8,150
F7 Muxes	1	1	16,300
Bonded IOB	30	0	106
BUFGCTRL	1	0	32

Table 32. Utilisation of the implemented design (Vivado 2018.3)

As Figure 20 shows, the sub-hierarchy u_muldiv holds 54 of the 62 LUTs and all 18 flip-flops. The flip-flop attribution is exact: the single-cycle datapath is purely combinational and contributes no registers, while every flip-flop in the ALU exists to hold the acc, q, m registers (12 bits), the 2-bit step counter and the four state/status bits (running, mode_div, done, div_by_zero) of the sequential multiply/divide unit: 12 + 2 + 4 = 18. The LUT attribution is less exact, because Vivado assigns logic that it merges across module boundaries (for example the operand multiplexers of the shared adder) to the hierarchy that drives it.

A cross-check with an independent open-source flow (Yosys, 7-series mapping, flattened design; an estimate, not a Vivado result) is given in the table below. The multiply/divide feature adds about 51 LUTs and 18 flip-flops to an ALU of about 23 LUTs, that is roughly two thirds of the LUTs plus all the registers, while muldiv_unit by itself contains about 32 LUTs. The qualitative conclusion therefore stands: MUL and DIV cost registers and a few multiplexers, not a second arithmetic unit.

Variant	LUTs	Flip-flops
Full ALU (10 operations)	74	18
ALU without MUL / DIV	23	0
Difference (cost of MUL / DIV)	+51	+18
muldiv_unit alone	32	18

Table 33. Cross-check of the area cost of MUL / DIV with Yosys (estimate, not a Vivado result)

8. Flow Explanation and Conclusion

8.1. Flow

Adding the nine design sources and the testbench separated, from the first step, the hardware description Vivado would turn into a chip from the testbench that only exercises it in simulated time. Running the behavioural simulation then compiled every file, connected the modules exactly as alu_4bit instantiates them, and stepped through simulated time with no knowledge yet of the target FPGA, producing the PASS line in F3. Opening the elaborated design (F4) turned the Verilog into a generic circuit, letting me confirm visually that the seven sub-modules and their connections matched what I had written. Running synthesis then mapped that circuit onto the real building blocks available on an Artix-7 (F5); and the utilisation report (F6) finally measured the concrete cost in LUTs and registers, confirming that the sequential multiply/divide unit holds all the registers and most of the LUTs.

8.2. Conclusion

This project designed, implemented and verified a 4-bit ALU in Verilog that performs ten operations, including multiplication and division built by shifting and reusing the ALU's own adder rather than by instantiating a dedicated multiplier array. The design was split into nine single-purpose Verilog modules, documented with a port table, a truth table from real simulation, the corresponding waveform, and the complete source code for each one; the file hierarchy and the inter-module signal connections were listed explicitly; and the whole design was verified exhaustively by a self-checking testbench covering all 4,096 opcode and operand combinations plus a set of protocol scenarios, ending in a printed PASS line with zero errors. Taken through the complete Vivado flow, the finished design occupies 62 LUTs and 18 flip-flops on an Artix-7 device, with the largest share of that cost attributable to the multiply/divide feature - exactly the area-versus-time trade-off the design set out to demonstrate.